Slot-Chain: A Preconfirmation Protocol

Audited Design Candidate — v2.13

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. After an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. Recovery is an episode of renewable, objectively expiring rounds, so a prover outage does not make its target stale; as with every validity rollup, producing the next proof still requires a reconstructible canonical prestate. This revision separates proved outputs from L1 close metadata, replaces the unauthenticatable queue hash chain with a range-provable Merkle vector, makes forced-message execution Ethereum-valid, and specifies bounded admission, data, reward and migration state. It is an audited architecture candidate, not yet an implementation-ready specification or production authorization: the missing executable profile and the proving, gas, economics and adversarial gates in §13 remain mandatory.

Contents

1 Status, claims and exclusions

Normative words MUST, MUST NOTand SHOULDhave their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness, a sound validity-proof system, availability of a canonical prestate reconstruction package as defined below, and—for kind-1 credits only—safety/finality of the supported source domain and its profile-authenticated, append-only Bridge registry, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state while the canonical prestate and any unexpired forced bytes remain reconstructible.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; kind 0 is executed if valid bytes are supplied before its bounded validity or consumed-and-discarded otherwise, while durable kind 1 is always applied idempotently.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. `L_LEASE` is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than `T_INCLUDE_MAX_SECONDS`, failure of L1 finality, or a broken proof system is outside the model.
- A supported source domain finality failure or unauthorized change to its profile-pinned Bridge registry is outside the kind-1 safety claim. It cannot authorize a kind-0 user envelope.
- A source message is not yet an admitted forced envelope. A preparer must obtain one fresh bounded source-state witness before creating destination-L1 `PENDING`; source emis-

sion alone is not reported as forced-queue acceptance. After emission the immutable source credit record can be re-proved beneath any later authenticated source header.

- A state root is a binding commitment, not an erasure code. If every copy of the canonical state trie, executed bodies, runtime-bytecode preimages and required witness data is destroyed after Ethereum's applicable data-retention horizon, a new prover cannot reconstruct the preimage from L1 roots. Arbitrary-duration recovery after total canonical-data loss is therefore not claimed.

2 Actors and trust model

Builder.
A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.
The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.
Any address. It supplies a validity proof and names an optional reward beneficiary in the proof statement. Proof validity, not identity, authorizes transition.

Canonical archive.
Any untrusted, replaceable source of canonical L2 bodies, state-trie nodes and runtime-bytecode preimages sufficient to reconstruct the current prestate. Each bytecode object is addressed by its legacy-Keccak `codeHash`; this includes code reached through proxy, beacon or delegation indirection. Content is verified against the L1-canonical block/state roots, so correctness does not require trusting the source; liveness requires at least one complete copy to remain reachable.

Forced-message sender.
Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

Bridge source domain.
For kind 1 only at launch, settlement Ethereum, identified by chain ID, genesis hash, registry address and immutable `bytes32` registry namespace. Its EIP-2935-authenticated state and append-only Bridge-epoch/credit records are proven to the same L1. Registry governance is an external safety dependency; a current resolver lookup never substitutes for the immutable emission record.

Source proof archive.
Any untrusted, replaceable source of the content-addressed fresh header, account/storage trie and runtime-code objects needed to prepare a kind-1 credit. Objects are root/hash verified and the archive has no protocol authority. An orphaned preparation uses a newly generated witness against a later header; it never depends on keeping an expiring emission-header receipt proof.

L1.
Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and

forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within `T_DEPTH_MAX=900s`; a longer L1 stall suspends, rather than violates, the L2 recovery clock. Archive operators have no protocol key, seat or exclusive API. Anyone may mirror or serve the same content-addressed data. This is an availability assumption, not an authority assumption; it limits long-horizon cold-start recovery but does not weaken proof soundness or permissionless submission.

3 Clock domains and canonical state

Before activation, Settlement is in `PREACTIVE`; candidate submission is disabled and slot arithmetic saturates. L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

`currentL2Slot=max(0,block.timestamp-GENESIS_TIMESTAMP).`

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations MUST NOTsubstitute one clock for another. `settlementChainId` is the L1 chain executing Settlement and domains every L1 contract commitment; `l2ChainId` is the EVM chain ID enforced for L2 transactions and execution. They are independent immutable launch constants and are both explicit in every cross-domain proof statement and builder header.

The L1 contract stores a proved core and local L1 metadata:

```
CanonicalCore = (l2BlockNumber, tipHash, tipSlot, stateRoot, messageCursor,
                 winningDataCommitment, nextBaseFee, nextExcessBlobGas)
Canonical = (CanonicalCore core, uint64 canonicalizedAtBlock)
```

The proof binds `baseCanonicalHash`, the hash of both stored objects, but outputs only a new `CanonicalCore`. Settlement compares every base field with storage, verifies the proof, writes the explicit proved core, and only then sets `canonicalizedAtBlock=block.number`. A prover is therefore never asked to predict the L1 block in which its transaction will land. No code attempts to read or store `blockhash(block.number)`, which does not exist during that block.

`l2BlockNumber` is the canonical EVM height, distinct from the one-second slot because slot gaps are legal. A candidate with `blockCount=n` proves `endL2BlockNumber=base.l2BlockNumber+n`; every executed header uses the next consecutive EVM block number. While the deployed `ICheckpointStore` uses `uint48`, launch and every transition additionally require `endL2BlockNumber<=UINT48_MAX`; changing that interface and bound is a protocol-version upgrade. The canonical event always emits `(l2BlockNumber,tipHash,stateRoot)` for bridge checkpointing. `nextBaseFee` and `nextExcessBlobGas` are the exact fork-rule outputs needed to construct the next header without retrieving a prunable historical parent body/header. The proof derives them from the last executed header; Settlement stores them but does not calculate fork arithmetic.

A candidate has one prior-block anchor. Normal activation stores the hash of its earlier arm block using native `blockhash`; candidate submissions hash the supplied execution header to that stored value and use MPT proofs, including the historical forced-queue root/count. EIP-2935 is used later to authenticate that arm hash for equivocation evidence. A recovery round instead stores `(block.number-1,blockhash(block.number-1))`, directly stores the current

queue root/count, and requires that anchor to reach depth `F_L1`. The EVM does not expose the parent timestamp. At candidate submission Settlement hashes and parses the supplied RLP parent header, requiring its number/hash to equal the frozen pair; its state root and timestamp become verifier-statement fields but are not separately frozen scalars. Current queue state is never claimed to be part of the parent's historical state root. An L1 reorg rolls the stored canonical state and anchor back together. Initial values and bridge cursors are migration inputs in §11.

Every state-mutating external entry point begins with `sync()`. If `sync()` changes mode or commits a mature normal candidate, the wrapper MUSTreturn `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the sync transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a `uint192` bond not exceeding `BOND_MAX`, a monotonic `uint64registrationIndex`, an `effectiveL2Slot`, and separately reserved `L_LEASE[window]` tranches. Address zero is permanently `VACANT`. Duplicate live addresses and reuse while any prior generation remains active or liable are forbidden. Once the old generation's last evidence/reorg deadline has passed, all tranches are terminal, its liability leaf is safely cleared and no evidence can still land, the address may register again with a fresh `registrationIndex`. No permanent address tombstone mapping exists. With `currentWindow=floor(currentL2Slot/384)`, a reservation is accepted only for `currentWindow<=W<=currentWindow+MAX_TRANCHE_AHEAD_WINDOWS`, where the initial ahead bound is 16; an exit request permanently disables new reservations for that generation. Only a present *active* generation with no exit request and no tombstone may execute `FREE->RESERVED`; a replacement or exit atomically closes reservations before moving the old generation into liability. Liability records may only be slashed or released, never extended with a new reservation.

The active table has exactly 64 indexed cells. On accepted registration, `effectiveL2Slot= currentL2Slot+384*ENTRY_DELAY_WINDOWS`; the stored value, not a later recomputation from a window boundary, is authoritative. Thus a registration becomes effective only after `ENTRY_ DELAY_WINDOWS=8`. If the table is full it may replace only the smallest-bond effective live entry, breaking ties by greatest registration index, and only with a strictly greater bond. At most `MAX_ GENERATION_MOVES_PER_WINDOW=4` replacements or effective exits are processed. Exit requests receive a monotonic sequence and mature at `requestWindow+EXIT_DELAY_WINDOWS`. Every replacement path first processes the oldest matured exits by `(matureWindow,exitSequence)` within the window's remaining movement budget; if any matured exit remains, replacement rejects. Anyone, including the exiting builder, may run this bounded maintenance. Thus transaction ordering cannot starve an eligible exit. A displaced generation is moved, with its tranche root and bond, into the fixed `MAX_LIABILITY_GENERATIONS=1,072` liability ring; an occupied ring cell is never overwritten before its release predicate holds. Registration therefore rejects, rather than losing slashable history, when the replacement budget or ring is full.

The six-level `registryRoot` over the 64 active cells commits each cell's mutable tranche root and therefore changes on reservation transitions. It is used only by schedule sealing. A sepa-

rate eleven-level admissionRoot over 2,048 fixed positions contains stable generation identity for the 64 active and 1,072 liability records plus canonical empty padding; its leaves do *not* contain tranche roots. Only generation admission, movement, tombstoning, slashing or release increments admissionVersion and updates admissionRoot. A FREE/RESERVED/LIABLE tranche transition updates the tranche and registry roots but cannot invalidate signed blocks or proofs by changing the admission pair. The admission root retains displaced scheduled keys. Normal activation and every recovery round freeze the pair (admissionVersion,admissionRoot). Each scheduled signer carries an inclusion proof for a non-tombstoned record effective at that L2 slot; a slash cannot retroactively invalidate an already frozen window. Outside an already-open normal window or live recovery round, a tombstone makes sealed future tickets at or after its effective slot return VACANT; inside that frozen candidate context the stored admission pair remains authoritative until close/commit/expiry.

Admission positions are canonical: active cell i is always position i for $0 \leq i < 64$; liability ring slot j is always position $64 + j$ for $0 \leq j < 1,072$. A generation move uses j= movementSequencemod1072. The transaction first proves/releases the old occupant, writes the moved generation at that same slot, clears its active cell or writes the replacement, increments movementSequence, then updates both affected roots and increments admissionVersion; the new admission pair is published atomically. Reservation-only mutations publish only the registry root and leave that pair byte-for-byte unchanged. There is no first-free search, compaction or caller-chosen liability position.

Eligibility is evaluated before ranking. A cell's effective L2 slot must be no later than the source-derived L2 slot, its fully reserved tranche for W must be proven, and its tombstone-effective L2 slot must be later than the source. All 64 registry cells are supplied; only present cells supply tranche-ring proofs at Wmod512. A canonical EMPTY tranche leaf is valid input and makes the cell ineligible; it need not claim window W. A nonempty leaf with a different stored window is likewise ineligible, not a seal revert. Only an exact RESERVED leaf with stored window W and the full L_LEASE amount is eligible. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. Supplying proofs only for purported winners is invalid because it cannot prove omission-free ranking.

An exit request becomes eligible for processing only after EXIT_DELAY_WINDOWS=MAX_LIVE_WINDOWS; it may wait behind the same bounded FIFO of older matured exits, and the bond remains escrowed until every generation and tranche is releasable. A tranche follows FREE->RESERVED(W)->LIABLE(W)->RELEASED or RESERVED/LIABLE->SLASHED. Every candidate block slot must be at least the normal window's frozen minimum (or the recovery round start), not merely its tip. Let windowEndSlot(W)=384*(W+1)-1. Release requires windowEndSlot(W)<globalMinReferencedSlot, no stored best or active round references it, and the absolute evidence and L1-reorg deadlines have expired. At that point maintenance atomically marks the schedule cell EXPIRED and may release its tranches; an unexpired sealed record is never overwritten. evidenceDeadline(W)=GENESIS_TIMESTAMP+384*(W+1)+EVIDENCE_DELAY_SECONDS; evidenceReplayDeadline(W)=evidenceDeadline(W)+REORG_MARGIN_SECONDS is both the contract's last admissible evidence time and the tranche's absolute release boundary. Evidence intended to survive a reorg must first be submitted by evidenceDeadline(W). globalMinReferencedSlot is the minimum of the current dynamic floor, an open normal window's frozen floor, an active round's start and the first slot of any stored best (absent values are ignored). Slot indices are never compared with timestamps or window numbers.

The liability-capacity proof is a launch invariant, not an assertion by inspection. A generation

moved in window C can have no reservation beyond C+MAX_TRANCHE_AHEAD_WINDOWS. Therefore

```
MAX_LIABILITY_RESIDENCE_WINDOWS =
  MAX_TRANCHE_AHEAD_WINDOWS + 1 +
  ceil((EVIDENCE_DELAY_SECONDS + REORG_MARGIN_SECONDS) / 384) + 2
```

must be strictly less than MAX_LIVE_WINDOWS=268. The two extra windows cover boundary ordering and bounded maintenance. At four moves per window, the ring position reused 268 windows later is consequently releasable. Each generation maintains an exact unreleased-tranche counter, so release and reuse do not scan 512 leaves.

4.2 Authenticated snapshot

For aligned window W, let its start timestamp be GENESIS_TIMESTAMP+384W. The target is the greatest BEACON_GENESIS_TIME+12k not later than eight L1 epochs before that start; L2-genesis alignment is irrelevant. Define sealDeadline(W)=windowStart-H_LOOK. A seal is valid only when its carrier execution block has F_FINAL L1-block depth and block.timestamp<sealDeadline(W). Beacon slots are not used as a surrogate for execution-block depth. At or after the deadline an unsealed window is permanently all-VACANT; a late transaction cannot change it. The contract performs this procedure:

1. The contract itself, not caller data, queries EIP-4788 for $j = 1, \ldots, 64$ at q=targetTimestamp+12j; missing timestamps revert and are skipped. The first successful query identifies the *carrier* execution block. EIP-4788 returns its parent beacon-block root, not the carrier root.
2. For the first successful query the caller supplies the carrier execution header. EIP-2935 authenticates its hash and block number; the header timestamp equals q and its parent_beacon_block_root equals the EIP-4788 result. Fork-specific SSZ branches in that parent authenticate its slot and one execution payload's blockHash, stateRoot, prevRandao and timestamp. The parent slot must be at or before target, its payload timestamp must equal its beacon-slot time, and its payload block hash must equal the carrier header's parent hash. No carrier observed by all 64 contract calls means all VACANT. If any call succeeds, a malformed header, a later parent, or a fork/gindex mismatch reverts without recording state; adversarial witness bytes can never manufacture a vacant seal.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header and registryRoot. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to registryRoot. Filter eligibility and order the entries, then store the byte-exact 64-leaf entryRoot and seed. Every *present* active cell supplies its tranche leaf and nine-sibling proof at index Wmod512. An absent registry cell has trancheRoot=0, supplies no tranche proof and is synthesized ineligible. For a present cell, canonical EMPTY, FREE, or a nonempty leaf for a different stored window is valid but ineligible; only exact RESERVED(W) with amount=L_LEASE is eligible. Proofs occur in ascending cell-index order, and extra/missing proofs reject. The fixed cell count proves completeness with one MPT path rather than 64 independent storage paths. The carrier satisfies currentExecutionBlock-carrierBlockNumber>=F_FINAL; the source payload is necessarily older.

sealWindow is permissionless. An optional maintenance distributor may pay its event in a later transaction. A missing seal fails closed to VACANT; it cannot give an address an unauthorized

ticket. The snapshot age plus 64-slot carrier scan, finality and seal margin fits the eight-epoch geometry and remains far below EIP-4788's 8,191-slot history window. Consensus constants include the Deneb SSZ generalized indices; an L1 fork that changes them requires a protocol-version upgrade, never runtime guessing.

For Deneb, Electra and Fulu, the generalized index from `BeaconBlock` root to `slot` is 8 and to `body.execution_payload` is 201. Within the composed tree, payload `state_root`, `prev_randao`, `timestamp` and `block_hash` are 6,434, 6,437, 6,441 and 6,444 respectively. The seal input names the L1 fork digest; only these configured digests are accepted. Gloas or any fork that changes the container requires new constants, fixtures and a protocol-version upgrade before its activation.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$\begin{aligned} seed_W &= H(\texttt{"slot-chain-seed-v1"} \| u256(settlementChainId) \\ &\quad \| u256(protocolVersion) \| u256(W) \| bytes32(prevRandao)), \\ qTie_i &= H(\texttt{"slot-chain-quota-tie-v1"} \| seed_W \| u64(regIndex_i)), \\ sTie_{p,i} &= H(\texttt{"slot-chain-slot-order-v1"} \| seed_W \| u16(p) \| address20(i)). \end{aligned}$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing `bond/(allocation+1)`, with `Q_MAX=76`. Quotients are compared by cross multiplication; a `uint192` bond times `Q_MAX+1` cannot overflow `uint256`. Equal quotients use smallest `qTie`. Remaining tickets are `VACANT`; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves `remaining[a] <=otherRemaining+1` for every real address. Choose the feasible value with smallest `sTie`; `VACANT` may repeat. The executable model and its golden vectors are normative test fixtures.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs exactly:

```
(settlementChainId, l2ChainId, protocolVersion, verifyingContract,
 slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
coinbase, tier, contextId, admissionVersion, admissionRoot,
 episode, recoveryRevision, recoveryId)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
```

```
uint256 settlementChainId,uint256 l2ChainId,uint256 protocolVersion,
address verifyingContract,
uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
bytes32 dataManifestRoot,address coinbase,uint8 tier,
bytes32 contextId,uint64 admissionVersion,bytes32 admissionRoot,
uint64 episode,
uint64 recoveryRevision,
bytes32 recoveryId)
```

The hashed ASCII has one space between each field type and field name and no other whitespace; displayed line breaks, indentation and spaces after commas are typography only. The exact single-line literal and a standalone Keccak implementation are in `commitment-model.py`. `TYPEHASH` is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

```
0xee6a8c8e31e8245cd527869508f6e464
d6084893991203876f734d1855aed87c
```

The EIP-712 domain is `("SlotChain","2",settlementChainId,verifyingContract)`. The explicit `l2ChainId` is the EVM `CHAINID` used by raw L2 transactions. Signatures are exactly 65-byte `r||s||v`; `v` is 27 or 28, s is low, and zero or recovered-zero addresses reject. EIP-2098 and EIP-155 transaction-style `v` values are not accepted. Tier 1 requires

```
contextId = H("slot-chain-normal-context-v1" || baseCanonicalHash ||
              u64(admissionVersion) || admissionRoot ||
              u64(armBlockNumber) || armBlockHash)
```

and its execution anchor is exactly that arm block. `episode=recoveryRevision=0` and `recoveryId=0`; tier 2 requires the exact live round values and `contextId=recoveryId`. The unsigned tier 3 uses that same recovery context. Consequently semantically unused bytes cannot create false equivocation evidence. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition. The signed `bodyRoot` covers only canonical discretionary transaction bytes; the proof derives the full Ethereum transactions root after adding the profile-defined system and valid forced transactions.

5.2 Three validity tiers

For every tier and every block, the circuit requires the EVM header `timestamp=GENESIS_TIMESTAMP+slot` with checked addition, and derives `blockHash` from that exact header. The signed relative slot can therefore never disagree with EVM time, forced-expiry classification, or the canonical tip clock. Every signed block also carries the exact frozen `admissionRoot`; the circuit checks it together with `admissionVersion`.

1. **Normal signed.** Every block is signed by its scheduled builder under the normal window's frozen `(admissionVersion,admissionRoot)`. Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Every block is at or above the window's frozen `minAdmissibleSlot`; parent gaps are

at most `G_MAX=3,600`, equal to the objective final-lag trigger and within the 12-window schedule-proof cap.

2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode and current recovery revision, and may cross an unbounded time gap. Every block still requires its scheduled builder signature under the current recovery round's frozen admission pair. Every slot is at or after that round's `roundStartSlot`.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the current round's frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the current recovery round and proved execution result.

Every candidate uses exactly one `batchAnchor`; every block repeats that anchor and frozen `forceRoot/forceCutoff`. For a normal candidate the supplied RLP execution header hashes to the EIP-2935 value, its timestamp is no later than the first L2 block timestamp, and MPT proofs under its state root authenticate the historical forced queue and any other contract state. For recovery, the RLP header authenticates the stored parent number/hash and derives its timestamp and state root; the queue pair is authenticated directly by the round fields and is not asserted against that header's state root. Requiring one anchor, not up to 4,096 independent anchors, bounds both circuit and L1 work. Cutoffs therefore do not change within a candidate; the first blocks drain the frozen FIFO prefix and later blocks see an empty prefix.

The circuit verifies parent linkage, strict slot monotonicity, schedules, signatures for tiers 1–2, version binding, anchor/header/MPT authentication, forced-prefix ordering, execution, exact body/data coverage and public outputs. It rejects more than 4,096 blocks, 12 schedule windows, 16 sealed sessions, 2,100 referenced blob records, 256 forced items, 4 MiB forced bytes or 80 million total accounted forced gas per candidate. These candidate-level caps are independent of the per-block caps.

5.3 Verifier statement ABI

The L1 verifier accepts `verify(proof,uint256[2]digestLimbs)`. Settlement constructs the following Solidity struct in exactly this order and computes `statementHash` as legacy Keccak of the ASCII domain `"slot-chain-statement-v1"` followed by `abi.encode(S)`:

```
S = (uint256 settlementChainId, uint256 l2ChainId, uint256 protocolVersion,
     bytes32 executionProfileHash, address verifyingContract,
     uint8 tier, bytes32 baseCanonicalHash,
     bytes32 candidateCommitment, uint64 blockCount,
     uint64 firstSlot, bytes32 tipHash, uint64 tipSlot,
     uint64 endL2BlockNumber, bytes32 endStateRoot, uint64 endCursor,
     bytes32 winningDataCommitment, uint64 nextDueAt,
     uint256 nextBaseFee, uint64 nextExcessBlobGas,
     uint64 anchorNumber, bytes32 anchorHash,
     bytes32 anchorStateRoot, uint64 anchorTimestamp,
     bytes32 forceRoot, uint64 forceCutoff,
```

```
bytes32 forcedDescriptorCommitment,
uint64 startCursor,
uint64 admissionVersion,
bytes32 admissionRoot,
uint64 episode, uint64 recoveryRevision, bytes32 recoveryId,
bytes32 scheduleRootsCommitment, uint8 scheduleWindowCount,
bytes32 sessionRefsCommitment, uint8 sessionCount,
uint16 dataRecordCount, bytes32 executionOutputsCommitment,
address proofBeneficiary)
```

`digestLimbs[0]` and `[1]` are respectively the high and low 128 bits of `statementHash`; neither is reduced modulo the proof field. The circuit recombines them and proves the exact preimage. Settlement independently checks the immutable `protocolVersion->executionProfileHash` mapping, the base canonical state, current/frozen versions and recovery round, recomputes the bounded schedule-root and sealed-session-reference commitments from calldata, recomputes the bounded forced-descriptor commitment from authoritative queue storage, recomputes `winningDataCommitment` from the candidate and sealed-session commitments, requires `endL2BlockNumber=stored.l2BlockNumber+blockCount`, and constructs the new core from the explicit `(endL2BlockNumber,tipHash,tipSlot,endStateRoot,endCursor)` fields plus that recomputed value and proved `(nextBaseFee,nextExcessBlobGas)`. It rejects zero/ill-typed fields, cursor regression, or disagreement with the last proved block, then stamps `canonicalizedAtBlock=block.number` outside the proof. `candidateId=statementHash`. `nextDueAt` is authenticated inside the frozen queue as the boundary leaf's deadline, or `UINT64_MAX` when `endCursor=forceCutoff`. It is only a proof-consistency output. Normal submission additionally verifies the current-root boundary proof described in §7. After every commit and on every activation check, the sole authoritative live head value is `ForcedQueue.dueAt(canonical.messageCursor)`; no recovery-round or canonical-core scalar caches it. `proofBeneficiary` is a proof input, never builder-signed block data or coinbase. Copying proof bytes cannot redirect a later optional reward.

5.4 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and
- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications MUST NOTdisplay the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted transactions to the mempool.

6 Blob data authentication

Data is posted into publisher-owned, bonded sessions. A session ID is

```
H("slot-chain-session-v1" || u256(settlementChainId) || address20(contract) ||
  address20(owner) || u64(ownerNonce))
```

At most two sessions per owner and 1,024 globally may be live; each contains at most 2,100 records. Opening uses the first free ring cell after at most `MAX_GC_STEPS=8` deterministic expiry checks from a persistent cursor; callers may run bounded maintenance repeatedly. No call scans or clears all 1,024 cells. A full live ring rejects discretionary publication but cannot block the forced/escape lane. Dynamic rent, charged for the entire TTL and burned into local accounting, prices global occupancy. A Sybil willing to rent every cell can censor this optional DA market for one TTL; this is an explicit residual, not a protocol-liveness claim.

Only the owner may append, and a candidate may reference only a session made immutable by `sealSession`. Sealing fixes `(root,count,expiry)`; there is no unseal or append-after-seal operation. A candidate supplies at most 16 sorted unique `(sessionId,count,root)` references. At submission each must equal a live sealed session. Normal acceptance requires `expiry>=normalDeadline+REORG_MARGIN_SECONDS`; recovery requires `expiry>=block.timestamp+REORG_MARGIN_SECONDS`. Opening requires its expiry to cover proving, settlement and the reorg margin:

$$\begin{aligned} expiry &\geq now + P\_PROVE\_MAX + W\_SETTLE\_SECONDS \\ &\quad + REORG\_MARGIN\_SECONDS, \\ expiry &\leq now + DATA\_TTL. \end{aligned}$$

For every blob attached to `postData(session,manifests[])`, the contract:

1. obtains `versionedHash=BLOBHASH(i)` from the same transaction;
2. computes legacy Keccak over this packed fixed-width sequence:

   ```
   "slot-chain-data-fs-v2" || u256(settlementChainId) ||
   u256(protocolVersion) ||
   sessionId || versionedHash || fullBodyRoot || u16(blockOrdinal) ||
   u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
   chunkRoot || address20(publisher) || u64(validUntil)
   ```

   It interprets the digest big-endian and reduces it modulo the BLS scalar-field modulus to obtain z;
3. calls the EIP-4844 point-evaluation precompile with exactly `versionedHash32||z32||y32||commitment48||proof48`; and
4. appends the exact leaf from §A to the session MMR.

The proof receives blob polynomials privately, recomputes $P(z) = y$, and decodes the canonical blob codec. A discretionary body byte string is `u32(txCount)||(u32(txLength)||signedRawTx)*`; its root is `H("slot-chain-body-v1"||u32(byteLength)||bodyBytes)`. Blob payload is `u32(chunkByteLength)||chunkBytes||zeroPadding`, split into 31-byte chunks; each blob field element is `0x00||chunk31`. Exactly 4,096 elements are used, unused payload bytes are zero, and noncanonical encodings reject. A manifest entry binds `(blockOrdinal,chunkIndex,chunkCount,chunkByteLength,fullBodyRoot,chunkRoot,sessionId,recordIndex)`. For each block entries are in strict chunk-index order, cover exactly `0..chunkCount-1`, have `chunkCount<=MAX_CHUNKS_PER_BODY=9`, and concatenate to at

most `MAX_BODY_BYTES=1,142,748`. The circuit recomputes every chunk root, the full discretionary body root and then the full Ethereum transactions root. The signed `dataManifestRoot` is *per block*. Its leaves contain only that block's entries, use local positions `0..m-1`, repeat that block's candidate-wide `blockOrdinal`, and are sorted by chunk index. Candidate calldata concatenates these independently rooted lists in increasing block ordinal; no global manifest tree exists. Duplicate, extra, overlapping or uncovered data is invalid. `dataManifestRoot` is the exact per-block tree in Appendix A, not a free-form publisher value.

Data publication has no consensus-coupled reimbursement. A separate service may pay it, but empty payment state cannot revert canonical settlement.

6.1 Canonical reconstruction availability

Blob sessions make an in-flight candidate provable; their TTL and garbage collection are not permanent state archival. On canonical commit, full nodes must retain the canonical raw bodies, receipts, Ethereum trie nodes and every runtime-bytecode preimage needed to execute from `CanonicalCore.stateRoot`. Bytecode is mandatory because an Ethereum account leaf authenticates only `codeHash`, not the corresponding bytes. A reconstruction package is a content-addressed set of those objects plus the canonical block headers; each code object is keyed by `H(runtimeBytecode)` and must cover every account reachable during replay, including implementation code reached through proxy, beacon or delegation indirection. A consumer rejects any object whose transaction/body/header hash, trie path, bytecode hash or final state root disagrees with the L1-canonical commitments, so an archive can be anonymous and malicious without gaining safety authority.

Renewing a recovery round refreshes its L1 anchor, queue cutoff and proof target; it cannot invert a state root or recreate destroyed bytes. Therefore the 2,164-second protocol bound begins only once a prover has the canonical reconstruction package and any required unexpired kind-0 bytes. A warm full node satisfies this condition; a cold-start prover is bounded by data download plus proof time while at least one archive or Ethereum's applicable DA history still serves the package. If no complete copy exists, safety remains intact but progress stops. No governance action may replace the state root, skip the missing prestate, or call that condition a builder failure.

7 Settlement and recovery state machine

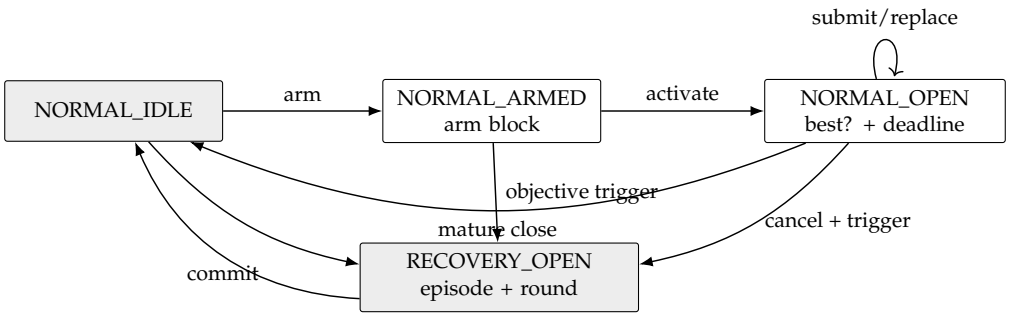


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal context and candidates

Normal context creation is two-phase. Anyone calls `armNormalContext()` in `NORMAL_IDLE`; the call begins with `sync()`, stores only `armBlockNumber=block.number`, and emits `NormalContextArmed`. It accepts no caller-supplied anchor or hash. In a later L1 block anyone calls `activateNormalContext()`. It again begins with `sync()` and requires the arm's block-number age to be from 1 through `MAX_ARM_AGE_BLOCKS` (255), reads `armBlockHash=blockhash(armBlockNumber)`, and hashes a supplied RLP header to that value. It then freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, `minAdmissibleSlot=max(0,currentL2Slot-DELTA_TIP)`, the current stable `(admissionVersion,admissionRoot)`, the arm header fields, and `requiredThrough=deadline+T_INCLUDE_MAX_SECONDS`. It derives the sole normal `contextId` and emits `NormalContextActivated`. No candidate is accepted while armed. If age exceeds 255 blocks, any caller may invoke `armNormalContext()` again; after its leading `sync()` it atomically replaces only the stale arm number with the current block, emits `NormalContextRearmed`, and returns `REARMED`. A nonstale arm cannot be replaced. Thus an abandoned arm cannot suppress normal operation.

Builders collect and issue tier-1 signatures only after the activation event. The execution anchor for every candidate is the exact arm block, whose hash and parsed header are already frozen; replacements hash their supplied RLP header to that stored hash and never repeat an EIP-2935 recency lookup. A reorg that removes the arm block necessarily changes the context. If the arm block survives but the activation transaction is reorged, reactivation intentionally recreates the same context: the builder must retain and honor its prior same-context signature. Evidence independently authenticates the arm hash as canonical through EIP-2935 and is accepted only while that history entry is available. The enforced evidence horizon is strictly shorter than 8,191 L1 slots. Thus neither a shallow reorg nor a builder-selected historical anchor can create false or free equivocation.

Activation is permissionless, reserves no builder, and does not block submissions. An empty activated window merely cancels at deadline, so an opener gains no censorship right. Every candidate first proves its frozen-prefix boundary in the circuit. It also supplies a canonical single-leaf Merkle proof at `endCursor` under the queue's current root/count, or proves `endCursor==currentCount`. Settlement verifies at most 32 sibling hashes and requires the resulting `currentBoundaryDueAt` to exceed `requiredThrough`. This prevents an old but valid anchor from hiding a post-anchor append. Because due times are monotone, one current boundary leaf proves that every item due through the landing horizon is consumed. Later candidates use the same base, deadline, minimum, admission pair and horizon. Their total order is lexicographic:

$$(\text{block count descending},\ \text{tip slot descending},\ \text{tip hash ascending}).$$

Only a strictly better candidate replaces `normalBest`; the stored best contains `endCursor` and the minimum expiry of every referenced data session. Provisional replacement changes no canonical state and pays nobody. At or after the deadline, `sync()` reads `ForcedQueue.dueAt(best.endCursor)` and commits only if that live boundary is greater than `block.timestamp` and `block.timestamp+REORG_MARGIN_SECONDS<=best.minDataExpiry`. Thus a late close cancels rather than committing a block that omits an already-due head, even beyond the assumed inclusion bound, or whose authenticated data has passed its resubmission margin. Data-session GC cannot evict a session referenced by the stored best; its own leading `sync()` clears an expired best first. Because `FORCE_DELAY>=W_SETTLE_SECONDS+T_`

`INCLUDE_MAX_SECONDS`, an append after open cannot violate the frozen acceptance horizon. Settlement then recomputes recovery causes against the new core.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot>canonical.tipSlot+DELTA_FINAL_LAG`; or
- the forced queue's stored `dueAt[canonical.messageCursor]` is no greater than `block.timestamp`.

A due forced head has precedence over an immature normal window, which is canceled. At a mature deadline, a best that consumes the required due prefix commits before causes are recomputed; an omitting best is canceled. No best is promoted across activation. The contract increments `episode`, records causes and the base-canonical hash, performs the SLA seat termination/burn at most once, then opens recovery round one. Termination and burn are fixed-size writes in Settlement-owned escrow accounting: no token transfer, registry callback or caller-controlled external call occurs on this path. A later sweep may dispose of recorded burns.

Every round freezes `roundStartSlot`, the preceding L1 block number/hash, current `forceRoot/forceCutoff`, current `(admissionVersion,admissionRoot)`, and `escapeSlot=max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)`. Its `expiresAt` is `GENESIS_TIMESTAMP+escapeSlot+DELTA_TIP`. The exact ID is:

```
recoveryId = H("slot-chain-recovery-v2",
               u256(settlementChainId), address20(contract),
               u64(episode), u64(revision), bytes32(baseCanonicalHash),
               u64(roundStartSlot), u64(anchorNumber), bytes32(anchorHash),
               bytes32(forceRoot), u64(forceCutoff), u64(admissionVersion),
               bytes32(admissionRoot), u64(escapeSlot), u8(causes))
```

At `block.timestamp>expiresAt`, and never before, `sync()` replaces the expired target with revision $r + 1$ and fresh round fields, then returns `SYNCED`. It does not change episode/base/cause, re-burn, terminate, promote or pay. The old proof is already inadmissible, so permissionless refresh cannot grief a still-live proof. New queue appends enter the refreshed cutoff. An envelope admitted during recovery receives `dueAt=max(enqueuedAt+FORCE_DELAY,lastDueAt,currentRound.expiresAt+1)`; therefore it cannot become due while a frozen round that omits it remains admissible.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the exact current stored canonical state;
2. bind current `episode`, `revision`, `recoveryId`, frozen admission pair, anchor and queue target;
3. land at least `F_L1` L1 blocks after its anchor, have first slot at least `roundStartSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;

4. consume the exact maximum forced-message prefix from the current cursor, bounded by the current round's frozen cutoff; and
5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block *number only*, clears the recovery episode and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

`ForcedEnvelope` is a tagged union. Kind 0 contains `(sender,nonce,l2ChainId,rawTxHash, byteLength,gasLimit,maxFee,validUntil,refundAddress,deposit)`. Admission canonically decodes the EIP-2718 transaction, requires its recovered sender and all duplicated fields to match, and requires `msg.sender==sender`; relayed admission needs a separately versioned EIP-712 authorization. Kind 1 contains:

```
(msgHash, srcChainId, sourceDomainId, srcEpoch, srcBridge,
 emittedAtBlock, srcBlockNumber, srcBlockHash,
 destChainId, srcOwner, destOwner,
 value, calldataHash, escrowId, byteLength, accountedGas,
 refundAddress, deposit)
```

Only the configured `BridgeInboxAdapter` may enqueue it and it has no expiry field. Source BridgeV2 locks the message value and fee under the exact `escrowId`; the destination adapter separately escrows its proof/queue deposit. Its only forced effect is an idempotent inbox/signal credit keyed by the immutable identifier

```
sourceDomainId = H("slot-chain-source-domain-v1" || u64(srcChainId) ||
  bytes32(sourceGenesisHash) || address20(epochRegistry) ||
  bytes32(registryNamespace))
creditId = H("slot-chain-bridge-credit-id-v3" || u64(srcChainId) ||
  sourceDomainId || u64(srcEpoch) || address20(srcBridge) || msgHash)
escrowId = H("slot-chain-bridge-escrow-v1" || creditId)
```

The registry exposes exactly two public immutable `bytes32` fields: `sourceGenesisHash` and `registryNamespace`. Its constructor ABI takes `(bytes32,bytes32)` in that order; both getters return `bytes32`, and either zero value rejects deployment. They are never dynamic bytes or integers. The deployed runtime hash commits the substituted immutable values, and the published `sourceDomain` vector in Appendix A pins the resulting packed encoding. arbitrary destination invocation and RETRIABLE processing remain later ordinary bridge operations. The two variants have separate byte-exact leaves.

The adapter is an exactly-once state machine `NONE->PENDING->QUEUED(index)`, keyed by `creditId`, not by `msgHash`. `BridgeEpochRegistry` is a profile-pinned non-proxy contract; those two constructor immutables and their exact ABI are pinned by the execution profile, and its code has no self-upgrade path. Replacing the registry requires a new address and creates a new source domain even if chain, epoch, Bridge address and message hash repeat. Destination protocol versions that retain the same source registry retain the same domain ID.

Launch accepts only when source and settlement chain IDs are equal. Let `sourceAge` be the current L1 block number minus `srcBlockNumber`. Preparation requires

```
F_L1 <= sourceAge <= MAX_SOURCE_WITNESS_AGE = 255
```

and reads the canonical hash for `srcBlockNumber` from Ethereum's EIP-2935 history contract and rejects unless it equals both `srcBlockHash` and legacy Keccak of the supplied canonical RLP header. Here `srcBlockNumber` and `srcBlockHash` identify this refreshable witness header, while `emittedAtBlock` is the immutable source-emission height. The adapter extracts the state

root from the authenticated header; no caller-supplied "finality certificate" or root is authoritative. The header is at most 2,048 bytes; the entire source witness is at most 524,288 bytes and 256 state-trie nodes, and verification receives at most 10,000,000 gas. Extra nodes, duplicate paths, noncanonical RLP, an expired history index or any limit breach reject. Supporting another source consensus requires a new protocol version with an equally byte-exact, bounded, audited finality verifier and is not part of the launch profile.

Source-header authentication, complete static-envelope validation, value escrow and one queue-capacity reservation atomically create an immutable `BridgeCredit` record and `PENDING` state. Under that source header, the state witness authenticates the non-proxy `BridgeEpochRegistry`'s immutable `CreditRecord(creditId)`. Source BridgeV2 creates this record in the same transaction that locks the exact message value/fee in `escrow[creditId]`, after checking `escrowId`; either both writes and message emission succeed or all revert. The registry recomputes `creditId`, requires the caller to be the unique live Bridge for `srcEpoch`, and permanently stores `sourceDomainId`, epoch, Bridge, execution hash, `emittedAtBlock`, message hash and every ownership/value/calldata/destination field. A duplicate credit ID rejects. The destination proof authenticates the record, its referenced immutable epoch and the exact descriptor. The registry record itself is the sole authoritative source signal: no external source SignalService slot is required. An event is an indexing aid only and a receipt proof has no consensus role. The logical record is exactly:

```
(bytes32 sourceDomainId, uint64 srcEpoch, address srcBridge,
 bytes32 bridgeExecutionHash, uint64 emittedAtBlock, bytes32 msgHash,
 uint256 destChainId, address srcOwner, address destOwner, uint256 value,
 bytes32 calldataHash, bytes32 escrowId)
```

The release profile pins its Solidity mapping slot and every packed member offset; no implementation may infer them from compiler defaults.

The destination's profile-pinned, non-proxy `SourceVerifierRegistry` retains an append-only `SourceDomainDescriptor` keyed by `sourceDomainId`. It contains the source chain ID, genesis hash, epoch registry address and namespace, registry runtime hash, constructor ABI, epoch/credit-record storage layouts, EIP-2935 parameters, BridgeV2 activation height, witness limits and historical verifier address/runtime hash. The registry recomputes the domain ID from those exact fields. Duplicate registration is idempotent only for byte-identical data; no entry can be deleted, replaced or redirected. Thus replacing source registry R1 by R2 creates D2 while the current adapter can still route an old D1 proof to R1 and its immutable verifier.

Only the profile-pinned delayed `ProtocolVersionManager` may register a domain or support entry, and only in the same transaction that activates an immutable protocol-version manifest containing the exact entry. Each release adds at most 64 domain/support entries, but the historical registry has no global lifetime cap. A support entry is keyed by `(sourceDomainId, bridgeExecutionHash)` and permanently stores the active protocol version, manifest hash, verifier address/runtime hash and `activatedAtBlock`. A later protocol version may omit an entry from its own finite manifest because the historical entry and verifier remain routable; it may never mutate their meaning. An unauthorized caller, inactive manifest, fresh registry address or conflicting duplicate rejects.

A Bridge-shaped clone or an app outside its live authorization epoch cannot create the source registry record. The adapter records the source epoch, domain, Bridge, emission height and witness header number and hash. Checking only the current resolver is forbidden. Because the domain, support, credit and epoch records are append-only, any later header satisfying the

age bound can replace the witness header without changing `creditId` or source semantics.

The source `BridgeEpochRegistry` retains every monotonically increasing `srcEpoch`, source Bridge, `bridgeExecutionHash`, activation block and deactivation block. Epochs are nonoverlapping. A successor and any shortening of its predecessor's initial `UINT64_MAX` end must be scheduled more than the source finality delay in advance; activation, elapsed intervals and past endpoints are immutable. The execution hash is

```
H("slot-chain-source-bridge-execution-v1" ||
  u32(topologyByteLength) || canonicalTopologyBytes)
```

where Appendix A defines the complete bounded grammar and dispatch reconstruction algorithm. The source registry or its immutable epoch controller owns every upgrade/selector authority. At emission BridgeV2 verifies its complete topology and terminal code hashes against its live epoch; the registry binds that epoch's immutable execution hash into the credit record. A later destination witness proves the record and epoch and matches the hash to the historical support entry; it does not reinterpret emission through the source's then-current proxy state. A topology change requires a future successor epoch. Its source activation block must be at least `PENDING_FINALITY_BLOCKS` after the destination profile containing that exact support entry became active. Source epoch scheduling requires that already-active entry and the full lead; predecessor shortening becomes effective atomically only if successor activation succeeds. Activation rechecks the same registry entry and inequality. BridgeV2 must reject emission under an epoch whose destination-support lead has not elapsed. Therefore destination support is active and final before the first new-topology emission, while all older emissions remain provable forever. At launch, source, destination and both registries share Ethereum, so the source controller reads this support state directly. Cross-chain support would require a separately reviewed finality bridge and is outside this profile. BridgeV2 checks that its own address, execution hash and epoch are the unique live entry before sending a V2 message. Thus a rotation has block-boundary semantics, same-block transaction ordering cannot change who was authorized at emission, and a later proof can still authenticate the old epoch. Admission also requires `0<srcChainId<=UINT64_MAX`, nonzero `sourceDomainId`, `0<srcEpoch<=UINT64_MAX`, `0<emittedAtBlock<=srcBlockNumber<=UINT64_MAX`, nonzero `srcBlockHash` and `msgHash`, `destChainId=l2ChainId` and a nonzero `srcBridge`; no narrowing cast or destination-chain inference is permitted.

`ForcedQueue.pendingReservations` is incremented in the same transaction that records `PENDING`. Preparation first validates every immutable append predicate—kind, authorization, source fields, gas, byte length, deposit and escrow—and requires `queueCount+pendingReservations<UINT32_MAX`. Every kind-0 append observes the same inequality. Thus `PENDING` owns one future leaf position and cannot be stranded by a capacity race or an envelope that will later fail a static check. Anyone may call `finalizeEnqueue(creditId)`. A Router `SYNCED` return leaves it `PENDING`; an `ADMITTED(index)` return sets `enqueuedAt=block.timestamp`, computes `dueAt` from that exact append time, consumes one reservation, increments queue count and changes the record to `QUEUED(index)` atomically. After static preparation, those are the only two outcomes. Repeated calls for the same `creditId` return the stored index; two historically authorized Bridge epochs with the same `msgHash` have different IDs and records. The upstream message is never reported queued merely because escrow became pending.

Before `PENDING_FINAL`, a permissionless preparer retains the source message fields and can regenerate a source package containing a fresh canonical RLP header, account/storage MPT

nodes for the registry credit and epoch records plus the registry runtime-bytecode preimage keyed by code hash. A consumer selects the append-only domain/support descriptors, then verifies every object against `srcBlockHash` and its state root. `PENDING_UNFINALIZED` is an operational label, not new contract state: if its L1 transaction is orphaned, anyone rebuilds the package against a later depth-final header and resubmits. The new header may change `srcBlockNumber/hash`, but the permanent record preserves `emittedAtBlock` and every credit field. There is no expiry race with the emission header. Exactly

```
PENDING_FINALITY_BLOCKS = F_L1 + ceil(REORG_MARGIN_SECONDS / 12) = 214
```

canonical destination blocks after its most recent inclusion does the operation become `PENDING_FINAL`; then the destination credit, reservation and escrow are sufficient. Wallets and source Bridge UI show a pre-final operation as *source observed*, never forced-queue accepted or source-independent.

Kind 0 requires a nonexpired `validUntil`, `validUntil<=enqueuedAt+MAX_FORCE_VALIDITY_SECONDS`, and `accountedGas=max(gasLimit,intrinsicGas,MIN_FORCE_ACCOUNTED_GAS)`. The initial validity horizon is seven days. Kind 1 uses the execution profile's fixed upper-bound InboxApply gas for its encoded credit. Both require `0<accountedGas<=5,000,000`, `0<byteLength<=131,072`, and a deposit covering execution, proof credit and permanent calldata/state costs. The adapter verifies kind-1 escrow and all static bounds before `PENDING`; failure leaves neither reservation, queue leaf nor partial credit.

Only `ActiveSettlementRouter` may append. It calls the active Settlement's `sync()` under a non-reentrant guard and returns `SYNCED` without appending if any transition occurred. Otherwise it computes `enqueuedAt=block.timestamp` and `dueAt=max(enqueuedAt+FORCE_DELAY, lastDueAt,recoveryExpiry+1)` (the last term is zero outside recovery), and appends atomically. A kind-1 append also consumes its prior reservation. `ForcedQueue` stores one authoritative `QueueDescriptor` per index. It is the complete fixed-width kind-0 or kind-1 leaf preimage in Appendix A, excluding only kind-0 `rawTx` bytes themselves but including their hash. Thus sender, nonce, chain IDs, byte/gas fields, fee, validity, refund address, enqueue/due times, deposit and every bridge-credit field remain durable. The Merkle leaf is always recomputed from this stored descriptor; admission atomically writes the descriptor, deposit/escrow accounting, append frontier and root or writes nothing. Settlement reads its `dueAt` directly for the canonical head and a stored best's live boundary; there is no callback, cached deadline or duplicated scalar. Missing indices return `UINT64_MAX`. Thus `dueAt` is nondecreasing and force-due/coverage checks are one authoritative O(1) read.

The queue is a depth-32 append-only Merkle vector with fixed capacity `UINT32_MAX` items, stored root, count and 32-node append frontier. Admission rejects at capacity; an upgrade must occur before exhaustion, but already admitted items remain live. A canonical DFS range multiproof authenticates the consumed leaves and, when present, the first excluded boundary leaf under the frozen `(forceRoot,forceCutoff)`. It supplies no fully covered subtree and no unused hash; at most 129 sibling hashes authenticate a 65-leaf block range. Skipping, reordering, changing the start index or claiming an early boundary changes the reconstructed root. Kind-0 raw bytes are enqueue calldata and their hash is in the leaf; blobs are forbidden on this availability path. The deliberately unused final tree leaf avoids a nonexistent height-32 frontier slot when the old index has all 32 bits set.

Each proved block consumes the unique maximum FIFO prefix satisfying all three bounds

simultaneously:

$$count \le 64, \qquad \sum byteLength \le 1{,}048{,}576, \qquad \sum accountedGas \le 20{,}000{,}000.$$

Every admitted item individually fits all three. The circuit exposes `(start,end,count, totalBytes,totalAccountedGas,forceRoot,forceCutoff,nextDueAt)` and verifies the range multiproof. Maximality means either `end=forceCutoff`, or the authenticated next leaf would exceed at least one remaining per-block cap; an early stop with a fitting leaf rejects. Candidate totals are independently capped as stated in §5. Launch enforces `ANCHOR_GAS_MAX+FORCE_GAS_BUDGET+SYSTEM_GAS_MARGIN<=L2_BLOCK_GAS_LIMIT` and the analogous witness-byte bound.

For the candidate's exact consumed interval, Settlement reads at most 256 descriptors plus the first excluded boundary descriptor when one exists under the frozen cutoff. Internal per-block boundaries are already the next consumed descriptor. It computes

```
H("slot-chain-force-descriptor-list-v2" || u64(start) ||
  u16(consumedCount) || u8(hasBoundary) ||
  (u32(index)||u8(kind)||u16(descriptorByteLength)||descriptorBytes)*)
```

as `forcedDescriptorCommitment`. The number of rows is the consumed count plus the zero-or-one boundary and is at most 257. `descriptorBytes` is the exact kind-specific packed sequence used by the leaf after its ASCII domain and index; its length is the unique constant for that kind. The circuit reconstructs every leaf, range proof, per-block maximum, candidate total and refund from these L1-authenticated descriptors. It additionally requires raw bytes for an unexpired kind 0 and the durable credit record for kind 1. An expired kind 0 needs neither historical calldata nor any unstored preimage.

Forced envelopes are auxiliary protocol inputs, not blindly copied into the Ethereum transaction list. Each block's transaction order is: (1) the exact profile-defined AnchorV4 transaction, (2) one deterministic `InboxApply` system transaction, (3) upfront-valid kind-0 raw transactions in FIFO order, then (4) discretionary transactions. Sequential pre-state classification puts expired, wrong-nonce, underfunded or underpriced kind-0 items only in `InboxApply`'s disposition vector; they have no Ethereum transaction, nonce or receipt. Upfront-valid kind 0 appears as an ordinary Ethereum transaction, whose success or revert has ordinary nonce/gas/receipt semantics. Kind 1 always produces its idempotent inbox credit inside `InboxApply` and never calls the eventual destination. The system calldata commits `(forceStart,forceEnd,(queueIndex,disposition,txIndex,resultHash)*)`; the circuit derives the transaction and receipt roots from this exact order. Unused deposit is later credited to `refundAddress`; failure there cannot change consumption.

Disposition bytes are fixed: `0=EXPIRED_NO_TX`, `1=NONCE_NO_TX`, `2=FUNDS_NO_TX`, `3=FEE_NO_TX`, `4=INCLUDED_TX` and `5=BRIDGE_CREDIT`. Classification tests in that order; the first applicable no-transaction reason wins. Codes 0–3 require `txIndex=UINT32_MAX` and `resultHash=0`. Code 4 requires the exact Ethereum transaction index; `resultHash` is legacy Keccak-256 of the raw signed EIP-2718 transaction bytes. This ordinary Ethereum transaction hash is known before execution. It is deliberately not a receipt hash: putting a later receipt hash in the earlier InboxApply calldata would create a fixed point through cumulative gas. Code 5 requires `txIndex=UINT32_MAX` and this exact durable credit hash:

```
H("slot-chain-bridge-credit-result-v6" || u64(queueIndex) || creditId ||
  msgHash || u256(srcChainId) || sourceDomainId ||
```

```
  u64(srcEpoch) || address20(srcBridge) ||
  u64(emittedAtBlock) || u64(srcBlockNumber) || srcBlockHash ||
  u256(destChainId) ||
  address20(srcOwner) ||
  address20(destOwner) || u256(value) || calldataHash || escrowId)
```

InboxApply recomputes it from the authenticated descriptor before including the exact tuple and `resultHash` in the one bounded SignalService batch below. SignalService, not Bridge, owns the persistent destination pin. Resolver rotation cannot retarget or orphan an existing credit. Unknown codes or noncanonical auxiliary fields reject.

AnchorV4 and InboxApply use a profile-defined, unsigned custom L2 system transaction type whose sender is injected by the post-cutover fork, not recovered from ECDSA. That type is invalid under every legacy fork, and the two reserved sender addresses are chosen with no known signing key. The circuit requires their exact profile transactions once each at indices 0 and 1 and rejects any ordinary, forced or discretionary transaction recovered from either sender. InboxApply also stores the last applied L2 block number and rejects a second application in that block. An ordinary caller therefore cannot invoke the privileged path before or after cutover.

The launch fork adds these exact SignalService operations:

```
struct InboxCreditV2 {
  uint64 queueIndex; uint64 srcChainId; bytes32 sourceDomainId;
  uint64 srcEpoch; address srcBridge; bytes32 msgHash; bytes32 resultHash;
}
markReceivedFromInboxBatch(InboxCreditV2[] calldata rows)
verifyInboxCredit(uint64 srcChainId, bytes32 sourceDomainId,
                  uint64 srcEpoch, address srcBridge, bytes32 msgHash)
    view returns (bytes32 creditId)
getInboxCreditSlot(bytes32 sourceDomainId, address srcBridge,
                   bytes32 creditId) pure returns (bytes32)
```

Only the immutable `InboxApply` predeploy may call the batch operation, and only while the L2 V2 activation latch is true. Every block calls it once, with exactly its zero to 64 code-5 dispositions in strictly increasing queue index and with no duplicate credit ID. InboxApply has already recomputed each result from the queue descriptor. SignalService validates widths and nonzero fields and performs all writes atomically. The slot is exactly

```
H("slot-chain-inbox-credit-slot-v1" || sourceDomainId ||
  address20(srcBridge) || creditId)
```

At `inboxResultHash[VERSION][slot]`, an empty value stores `resultHash` and sets the received bit at that same slot. A repeated row succeeds without writes only when the stored result matches; any conflict, ordering error or excess row reverts the whole batch before a write. `verifyInboxCredit` recomputes `creditId` and the slot, requires both result and received bit, and returns the ID. This is a new bytes32-domain ABI; it never overloads legacy `getSignalSlot(uint64,address,bytes32)`.

BridgeV2 introduces `SourceContextV2`. Its three fields are a `bytes32 sourceDomainId`, a `uint64 srcEpoch` and an `address srcBridge`. The complete V2 lifecycle ABI is:

```
sendMessageV2(Message)
  -> (msgHash, creditId, Message, SourceContextV2)
processMessageV2(Message, SourceContextV2, bytes proof)
  -> (Status, StatusReason)
retryMessageV2(Message, SourceContextV2, bool isLastAttempt)
failMessageV2(Message, SourceContextV2)
recallMessageV2(Message, SourceContextV2, bytes proof)
isMessageFailedV2(Message, SourceContextV2, bytes proof) -> bool
messageStatusV2(bytes32 creditId) -> Status
```

Here `Message` is the existing IBridge tuple and `msgHash=hashMessage(Message)`. Every mutating or proof-bearing function recomputes `creditId` from that hash and the context before any status, value or external effect; the read-only `messageStatusV2` view deliberately accepts the already-derived key. Source `sendMessageV2` obtains the context from its live epoch and creates the immutable CreditRecord. It emits `MessageSentV2(creditId,msgHash,SourceContextV2, Message);` destination changes emit `MessageStatusChangedV2(creditId,status)`. Both events' indexed fields and expanded tuple encoding are fixed by the profile. Destination status, retry state and value release are keyed only by `creditId`. Its empty-proof path calls `verifyInboxCredit`; its nonempty V2 path uses the append-only domain and support entries to prove the same source CreditRecord directly. Both paths therefore share one exactly-once status. V2 failure sends

```
H("slot-chain-bridge-failed-v2" || creditId)
```

and `recallMessageV2` proves that exact signal; no V2 failure, recall or view falls back to `msgHash`. The release profile pins every expanded ABI selector, event, status/failure storage slot and positive/negative lifecycle vector.

Post-activation source BridgeV2 sends only the CreditRecord for a new message, never the legacy `msgHash` signal, so the legacy proof path cannot process the same message a second time. Pre-activation messages remain on unchanged V1 functions and are never converted. InboxApply makes one bounded batch external call, and neither it nor SignalService invokes a message recipient. Legacy proof-based `proveSignalReceived` remains unchanged. AnchorV4 and InboxApply are profile-defined system transactions: their custom type, reserved senders, system nonces, zero beneficiary tip and fee treatment are consensus fork rules, not ordinary user accounts. Their gas and the batch loop still count against the block limit. Deployment and migration require upgraded, vector-tested SignalService and InboxApply bytecode. Authorizing an unreviewed caller is invalid.

An unexpired kind-0 item must reveal raw bytes hashing to its stored `rawTxHash`; the circuit decodes them and matches every duplicated descriptor field before execution. Once `validUntil` is strictly before the L2 block timestamp, the circuit emits the unique `EXPIRED_NO_TX` disposition from the authenticated durable descriptor without needing those bytes. Consequently a disappeared sender or pruned history can delay its own item only until the bounded validity horizon, never wedge the FIFO permanently. A user that wants execution remains an available source for its own signed bytes. The 2,164-second recovery target is conditional on unexpired head bytes being retrievable; the unconditional protocol bound adds at most `MAX_FORCE_VALIDITY_SECONDS`.

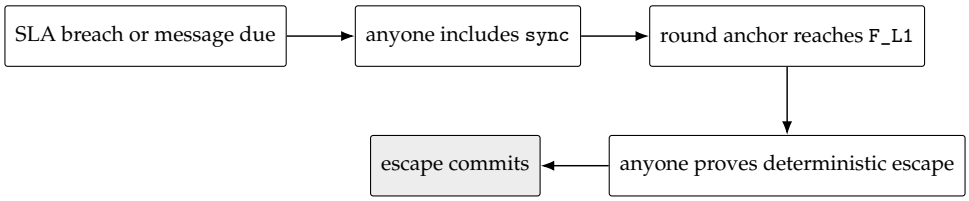
For a normal signed block, `forceRoot/forceCutoff` are authenticated by its L1 anchor. For recovery they equal the current round's directly stored pair. The circuit consumes only up to

that cutoff, so a later append cannot invalidate an already signed block or in-flight proof. All blocks in one candidate use the same cutoff. A due current head cannot be bypassed: `sync()` cancels an immature window, or closes a mature best only when its live boundary is not due, before opening recovery.

The 1,500-second force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic under a governance-frozen `executionProfileHash`. That profile commits the L2 fork rules; complete header-field derivation (gas limit, base fee, extra data, withdrawals/requests roots and randomness); the deployed AnchorV4 and InboxApply bytecode hashes; reserved senders, nonces and gas; AnchorV4 checkpoint calldata and `ancestorsHash` transition; and the transaction/receipt derivation above. For the initial profile the anchor is an EIP-1559 transaction from `0x0000777735367b36bC9B61C50022d9D0700dB4Ec`, gas limit 1,000,000, priority fee/value zero, max fee equal to the block base fee, empty access list, and the sender's sequential state nonce. It calls the frozen Anchor address with selector `0x523e6854` and ABI tuple `(uint48(anchorNumber),anchorHash,anchorStateRoot)` from the candidate's one batch anchor. Its deterministic protocol signature algorithm and the InboxApply transaction's corresponding fields are part of the profile vectors. Its timestamp is `GENESIS_TIMESTAMP+escapeSlot`, coinbase is the protocol sink and there is no discretionary body. A repeated batch anchor still performs the exact per-block ancestors transition. Changing the execution profile requires a delayed protocol-version upgrade and new full-block golden vectors; runtime guessing is forbidden.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen `ESCAPE_OFFSET=1,900s` (covering `T_DEPTH_MAX=900s`, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below `DELTA_FINAL_LAG=3,600s`. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit `DELTA_TIP=1,200s` when proof generation meets its bound.

9 Economics, slashing and payments

Event	Objective evidence	Effect
Same-context equivocation	Two distinct protocol-domain signatures for one key/slot/context	Slash that schedule window's tranche once; tombstone key; emit reporter entitlement.

Aggregator SLA breach	Final lag exceeds threshold after mature close	Terminate and burn penalty bond once; open recovery; promote standby if any.
Force-only activation	Due queue head	No aggregator penalty; open recovery.
Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

Equivocation evidence is idempotent per `(builder,window)`; separate windows slash separate tranches. `L_LEASE` is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

Evidence does not depend on retaining an obsolete `admissionRoot`. It contains two distinct low-*s* EIP-712 headers with the same recovered nonzero key, slot, tier, context ID, admission version, admission root, episode, revision and recovery ID. For tier 1 it recomputes the normal context from the same base, admission pair and arm-block anchor carried by both headers and authenticates that arm hash as canonical through EIP-2935; tier 2 recomputes the recovery context. The contract enforces at launch that the last tier-1 evidence/replay instant precedes expiry of that 8,191-block history entry. It supplies one proof under the signed admission root and one under the *current* admission root for the same address and `registrationIndex`; the latter must be that still-retained active/liability generation. Its current stored tranche root separately authenticates the window leaf, which is `RESERVED(W)` or `LIABLE(W)`. The two complete struct hashes must differ. Signing two conflicting protocol headers in one such context is slashable even if the key was not ultimately selected at that schedule position; builders must not sign off-ticket headers. The evidence transaction derives W from the slot, requires the tranche leaf's stored window to equal W, and lands by `evidenceReplayDeadline(W)=evidenceDeadline(W) +REORG_MARGIN_SECONDS`. Evidence available by `evidenceDeadline(W)` therefore has the full margin for reorg detection and re-inclusion; first publication delayed into the replay tail has no replay guarantee. Address reuse is forbidden while that retained generation exists, so the current proof cannot select a newer incarnation. This rule keeps evidence verification fixed-depth while the liability-capacity invariant retains every slashable generation through the replay deadline.

Canonical settlement performs no reward calculation, token balance write or external call. It always emits a candidate-committed event containing the ID, beneficiary and reward class. Best-effort protocol claims use a fixed `MAX_REWARD_RECEIPTS=256` ring, selecting `uint8(uint256(candidateId))`: if that cell is empty or its prior receipt is both claimed/expired and past the reorg margin, Settlement records the three fields, commit block and `claimed=false`; otherwise it only emits the event. Ring occupancy can therefore forfeit a reward but can never revert or delay canonical state. A separate distributor verifies this stored receipt, sets `claimed=true` before transfer, and pays at most the funded class cap before `REWARD_CLAIM_WINDOW=86,400s`. Logs alone are explicitly not on-chain-verifiable entitlements. Burns are never recycled into the same episode's bounty. Normal service/data compensation likewise cannot affect candidate validity or canonical commit.

10 Security and liveness analysis

Adversary/failure	Protocol result	Bound or residual
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator offline/byzantine	SLA activation terminates it; any prover uses tier 2 or 3.	At most final-lag threshold plus 2,164 s under stated inclusion/proof and head-data bounds.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.
All builders collude/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; preclose promises remain revocable.
Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Sessions are publisher-owned and bonded.	Attacker fills only its session; escape uses no blob session.
Missing/pruned kind-0 bytes	Unexpired head waits; after validity it is consumed as EXPIRED_NO_TX from its durable descriptor.	Adds at most seven days; kind-1 credit data is durable state.
Source registry/topology rotates	New domain/support manifest activates before source emission; old verifier routes remain immutable.	Old unprepared credits use fresh state witnesses through their retained domain descriptor.
Unauthorized source-support fill	Version manager rejects entries absent from the active delayed manifest.	At most 64 audited additions per release; no attacker-consumable lifetime cap.
Legacy call to installed V2 code	Shared L2 gate is false and the custom system transaction type is invalid.	First proved post-cutover Anchor activates once; all V2 modules recheck the gate.
All canonical prestate copies lost	No party can construct the next EVM witness from a root alone; proof generation stops without changing canonical state.	Explicit information-theoretic availability limit; restore any root-verifiable archive copy.
Transparent-mempool front-run	First valid recovery transaction wins.	Proof statement binds beneficiary, episode, revision and outputs; copying cannot redirect payment or state.
Payment-vault exhaustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.

L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after `T_INCLUDE_MAX_SECONDS`; no L2 protocol can force its L1 contract to execute.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

11 Implementation blueprint

11.1 L1 modules and bounded storage

- `BuilderRegistry`: 64 active cells, 1,072 liability generations, 512-leaf tranche roots, tombstones, replacement counters and the versioned `admissionRoot`. Every transition is fixed-depth or constant-size.
- `ScheduleOracle`: sealed window root/seed in a ring covering `MAX_LIVE_WINDOWS=268`. `MAX_EARLY_SEAL_WINDOWS=8` covers the earliest snapshot geometry. In window units the capacity is at least:

  ```
  MAX_EARLY_SEAL_WINDOWS + MAX_CANDIDATE_WINDOWS +
  ceil((W_SETTLE_SECONDS + DELTA_FINAL_LAG + DELTA_TIP +
        REORG_MARGIN_SECONDS + EVIDENCE_DELAY_SECONDS) / 384) + 2
  ```

  An entry is overwritten only after it is `EXPIRED`; the same transaction checks the release predicate and no stored normal best/round reference.
- `DataSession`: a 1,024-cell global ring, at most two live cells per owner, each with owner/nonce ID, 2,100-leaf MMR frontier, sealed root/count and expiry; no on-chain leaf array. Expired eviction advances at most eight cells per call.
- `ForcedQueue`: depth-32 append-only Merkle root/count/frontier, last due time, complete fixed-width per-index descriptors and deposits; an expired kind-0 descriptor reconstructs its leaf, prefix accounting and refund without historical calldata, while kind-1 credits remain durable. Only the router appends.
- `SourceVerifierRegistry`: non-proxy append-only domain and support mappings with immutable historical verifier routes. Only bounded entries from the manifest being atomically activated by ProtocolVersionManager may be added; there is no delete, redirect or age-proportional iteration.
- `BridgeInboxAdapter`: exactly-once NONE/PENDING/QUEUED records, escrow and one queue reservation per durable source credit. It selects source proof logic only through SourceVerifierRegistry and rejects while PREACTIVE.
- `Settlement`: PREACTIVE flag, canonical state, one normal best, recovery episode/round, seat pointer and a 256-cell best-effort receipt ring. Rewards and burn disposal are separate. No loop is proportional to chain age.
- `SettlementCheckpointAdapter`: the sole SignalService syncer. Its permissionless

`publish()` reads the current Settlement core, requires a strictly greater `l2BlockNumber`, checked-casts the globally bounded value to `uint48`, and writes exactly `(l2BlockNumber, tipHash,stateRoot)`. Failure or non-use delays bridge proofs but cannot affect canonical settlement; anyone may catch it up in one O(1) call to the latest core.

The non-authoritative `CanonicalArchive` is independently mirrored off-chain canonical bodies, receipts, state-trie nodes and runtime bytecode indexed by canonical block, state root and code hash. Every object is commitment-verifiable; no archive address, signature or availability vote appears in Settlement or the verifier statement.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

11.2 Historical authentication

Native `blockhash` plus a supplied matching RLP header authenticates a fresh normal arm block; EIP-2935 authenticates later normal-context evidence and schedule carrier headers. A recovery round captures its immediately preceding block hash with `blockhash(block.number-1)` and later hashes a supplied RLP header to that stored value; it never tries to read an unavailable parent timestamp, queries an expired history entry or mutates the meaning of a past header. Its queue pair is round storage, not an MPT claim. EIP-4788 plus SSZ branches authenticates the schedule's parent beacon/execution payload; its state root plus MPT proofs authenticates the registry. Deployments lacking either primitive require an equivalently verified oracle and separate review.

11.3 Executable execution profile

`executionProfileHash` is not a free-form governance label. Each `protocolVersion` maps immutably to one canonical CBOR profile whose bytes are published with the verifier key and hashed as `H("slot-chain-execution-profile-v1"||u32(byteLength)||profileBytes)`. The decoder rejects unknown keys, duplicate map keys, nonminimal integers, indefinite lengths and zero code hashes. The profile contains:

- settlement/L2 chain IDs, Ethereum genesis hash, fork digest, EIP-2935 address and activation block; SourceVerifierRegistry/ProtocolVersionManager addresses and runtime hashes; every append-only domain/support descriptor, manifest/activation binding and historical verifier; plus each source's nonzero `bytes32` BridgeEpochRegistry namespace, BridgeV2 activation, registry/epoch-controller runtime, constructor ABI, epoch/CreditRecord layouts and historical proof limits;
- gas limit, EIP-1559 elasticity/change denominator, blob target/update fraction, empty ommers/withdrawals/requests constants and exact derivation of base fee, blob fields, logs bloom, randomness, parent beacon root, requests hash and every post-fork header field;
- AnchorV4, InboxV2ActivationGate, InboxApply, SignalService and Bridge addresses, runtime code hashes, every expanded V1/V2 selector and ABI tuple; the unsigned custom system transaction type and unforgeable reserved senders; system nonce rules, fee exception, activation latch and gas ceilings;
- the exact AnchorV4 checkpoint/ancestors transition, forced disposition rules, InboxApply one-shot/batch transition, SignalService inbox-slot and idempotence storage, Bridge V2

status/failure/recall state and both proof paths; and

- at least six complete parent/prestate/input/transaction/receipt/header/ poststate vectors: empty escape, valid kind 0, expired kind 0 without bytes, kind 1 before and after a Bridge-epoch rotation, A–B–A address reuse with the same message hash, registry/domain replacement, pending-capacity race, delayed append, PREACTIVE kind-0/kind-1 rejection, legacy attempts to activate or invoke V2, first post-cutover latch activation, orphan/replayed pending under a refreshed witness, both source-age boundaries, durable-record absence, topology-support manifest authorization/activation/finality, historical domain routing, conflicting registration/removal, multi-credit batch ordering/conflict and a cap-limited multi-block prefix, plus invalid forged/expired EIP-2935 header, oversized/duplicate/unsorted topology and source witnesses, extra-reserved-sender, unauthorized clone, mismatched/conflicting source Bridge, conflicting result/failure hash, every V2 lifecycle method, cross-path replay and duplicate-InboxApply vectors.

For escape, parent hash/height/state and next fee/blob scalars come from the stored canonical core; timestamp/slot, L1 origin and forced inputs come from the round. Every remaining header field and both system transactions are a pure profile function of those values and the execution result. The circuit loads the profile selected by the public protocol version and proves its hash equals Settlement's immutable mapping. Launch tooling rejects a version lacking the CBOR artifact, verifier-key binding or complete vectors. Thus a profile change is a new protocol version and review, not runtime interpretation.

11.4 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: canonical tuple, mode/version, normal arm/activation/best/deadline/frozen admission, episode/round, seat termination/burn, tombstone/tranche state, forced cursor, data-session root/count, and reward eligibility. Off-chain workers index by `(settlementChainId,contract,blockHash, logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

11.5 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before activation, so first live sources are post-deployment and sealable. New Settlement begins `PREACTIVE`; registry, scheduling and data preparation may run, but every candidate and every kind-0/kind-1 forced-ingress or bridge preparation call rejects before creating escrow, a credit or a reservation.

The legacy Inbox does not share this design's queue or expose `freezeAndExport`. Migration therefore never pretends to adopt its blob-backed forced-inclusion records. A staged legacy upgrade first disables new legacy forced admission, then drains every admitted legacy record only through the legacy protocol's native consume/expiry/void rules. Existing records do not identify a general refund owner, so this design promises neither a fabricated refund nor conversion. Any separately governed claim mechanism must complete before migration and is outside this adapter. Activation is blocked until the audited native legacy predicates report an empty live queue and zero unsettled forced escrow. Only then is the new empty `ForcedQueue` placed behind `ActiveSettlementRouter`; its initial root/count/cursor and `dueAt(0)=UINT64_MAX` are golden migration values. Existing Taiko bridge messages remain on the legacy SignalService/Bridge retry path and are not silently converted into kind 1. Only new, explic-

itly escrowed inbox-credit messages use `BridgeInboxAdapter`; its finalized message manifest binds the existing message hash, source domain/epoch, source Bridge/execution identity, source emission block number and ownership/value/calldata fields and a separately refreshable source witness header number/hash, while destination invocation remains a later bridge operation.

After that drain, migration has three phases. First, before quiescence, the legacy L2 governance path deploys the profile-exact InboxApply runtime and upgrades the L2 SignalService with the bounded inbox-credit batch, setting its sole authorized caller to that InboxApply address. It also installs any profile-required AnchorV4 runtime and the Bridge logic whose empty-proof path queries SignalService's pinned credit instead of the live resolver. One non-proxy `InboxV2ActivationGate` is initialized false. InboxApply, the SignalService batch and every L2 BridgeV2 send/process/retry/fail/recall path independently require its `active()` value; only AnchorV4's one-shot custom-system transaction may set it. This is an ordinary finalized L2 governance block; no L1 transaction is claimed to mutate an imported L2 state root. Its transaction bytes, receipts and resulting account/storage values are golden profile vectors.

Second, anyone advances the old Inbox until its finalized transition is quiescent; the gate requires the next proposal ID to equal the last finalized proposal ID plus one, the old core's finalized block-hash field, an empty native forced queue and no unsettled escrow. The caller supplies the canonical RLP of that final L2 header and fixed-depth MPT account/storage proofs under its state root for the exact AnchorV4, InboxApply, SignalService and Bridge proxy/fork-router runtime code hashes; every EIP-1967 implementation/beacon slot and profile-defined fork-selector slot; the selected implementation accounts' runtime code hashes (which commit constructor immutables); InboxApply one-shot initial state; SignalService authorized-caller storage; each module's immutable gate address; and the gate's `active=false` state. The proof follows the exact proxy/router topology encoded in the profile and rejects an unlisted hop, delegate target or mutable beacon. Every terminal implementation and value must equal the immutable execution profile. The adapter requires its hash to equal that stored finalized hash, its `number<=UINT48_MAX`, and the legacy SignalService checkpoint at that exact number to equal its block hash and state root. This binds the height and state root that the current Inbox core does not store.

Third, the governance executor invokes the audited migration adapter. From that authenticated header and the frozen execution profile it constructs every initial field exactly:

```
l2BlockNumber       = header.number
tipHash             = H(RLP(header))
tipSlot             = header.timestamp - GENESIS_TIMESTAMP
stateRoot           = header.stateRoot
messageCursor       = 0
winningDataCommitment = H("slot-chain-migration-data-v1" ||
                          u256(settlementChainId) || u256(l2ChainId) ||
                          tipHash || stateRoot)
nextBaseFee         = profile.nextBaseFee(header)
nextExcessBlobGas   = profile.nextExcessBlobGas(header)
canonicalizedAtBlock = block.number
```

It rejects a pre-genesis timestamp, arithmetic overflow, missing fork-required header fields or any profile mismatch. It freezes old proposal/forced ingress, imports this complete core into new PREACTIVE state, verifies the new queue's golden empty root/count/cursor,

`pendingReservations=0`, `dueAt(0)=UINT64_MAX`, and an empty BridgeInboxAdapter with no `BridgeCredit`, `PENDING` record or escrow, atomically points router authority at the new queue/Settlement, changes new PREACTIVE to NORMAL_IDLE, and reconfigures or upgrades the L1 checkpoint SignalService so the immutable SettlementCheckpointAdapter is its authorized syncer, and stores the L1 activation commitment consumed by the first new AnchorV4 transaction. It does not write L2 code or storage. L1 kind-1 ingress is enabled only after the authenticated L2 code/authorization proofs and this L1 checkpoint authorization have all succeeded. That same atomic cutover is the first state in which kind-0 or kind-1 ingress is enabled. The first post-cutover L2 block must begin with the custom AnchorV4 transaction. The circuit authenticates the L1 cutover commitment, exact imported parent and execution-profile hash; AnchorV4 then flips the gate exactly once before that block's InboxApply batch. The custom transaction type is invalid under the legacy fork and its reserved sender has no ordinary signing key, so neither a legacy block nor an EOA can preactivate the latch. Any first block that omits, duplicates or mismatches this transition is unprovable. Subsequent blocks require the latch already true and cannot repeat activation. The operation contains only bounded non-reverting internal writes after its assertions; any revert rolls all contracts back. The adapter cannot cancel or convert legacy forced records; a nonempty legacy predicate simply blocks migration. Old bridge signals keep their original claim semantics, so the migration makes no unenforceable claim to enumerate all historical pending messages.

12 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
L2 height / timestamp	`uint48` checkpoint bound	Header number is consecutive and at most `UINT48_MAX`; header timestamp equals genesis plus signed slot.
`N_MAX`, `Q_MAX`	64 / 76	Six addresses fill 384 tickets.
`BOND_MAX`	$2^{192} - 1$	`bond*(Q_MAX+1)<2^256`.
Entry/exit/reservation ahead; moves	8 / 268 / 16 windows; 4 per window	Liability residence < 268; capacity 4×268.
`D_SNAP`, `F_FINAL`	8 / 2 L1 epochs	Strictly exceeds scan + finality + seal margin + lookahead.
`H_LOOK`	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
`G_MAX`	3,600 L2 slots	Tier-1 parent gap; equals final-lag trigger and fits within the 12-window proof cap.
`W_SETTLE_SECONDS`	1,200 s	Timestamp deadline, never L1 block count.
`DELTA_TIP`	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
`DELTA_FINAL_LAG`	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
`F_L1`; `T_DEPTH_MAX`	64 L1 blocks; 900 s	Stored recovery-anchor depth and assumed maximum time to reach it.

`ESCAPE_OFFSET`	1,900 s	At least depth-time bound + proof bound + margin.
`FORCE_DELAY`	1,500 s	At least settlement window plus `T_INCLUDE_MAX_SECONDS=120`.
Forced prefix	64 items; 1,048,576 B; 20m gas	Each item at most 131,072 B and 5m accounted gas; descriptors and one boundary remain durable.
Candidate caps	4,096 blocks; 12 windows; 1 anchor; 16 sessions; 2,100 records	Also 256 forced items, 4 MiB, 80m accounted gas.
Queue / range proof	depth 32 / at most 129 hashes	`queueCount+pendingReservations<=UINT32_MAX`; canonical contiguous range proof.
Source witness	64–255 blocks; 2,048 / 524,288 B; 256 nodes; 10m gas	Launch source is settlement Ethereum; a refreshable header is inside EIP-2935 history; header / total witness, state nodes and verification gas are independently bounded.
Source verifier release	at most 64 new entries/version; append-only	Only atomic activation of a delayed immutable manifest registers entries; the historical registry has no global cap, delete, redirect or reinterpretation.
Source epoch support	one immutable active-profile entry	New epoch waits `PENDING_FINALITY_BLOCKS` after its support profile activation; predecessor shortening is conditional on successful activation.
Max kind-0 validity	604,800 s	Bounds missing kind-0 payload delay; kind 1 is durable.
Body chunks	9 / 1,142,748 B	Per-body chunks / maximum discretionary bytes.
`DATA_TTL`	86,400 s	Greater than candidate, settlement, proof and reorg path.
`REORG_MARGIN`, `EVIDENCE_DELAY`	1,800 / 86,400 s	Data resubmission and finite liability retention.
`PENDING_FINALITY_BLOCKS`	214 L1 blocks	`F_L1+ceil(REORG_MARGIN_SECONDS/12)` after the most recent preparation inclusion.
Live windows / sessions	268 / 1,024	Fixed rings with safe-eviction predicates and bounded GC.
EIP history / max arm age	8,191 / 255 blocks	$255 + \lceil (W_{settle} + CLOCK\_SKEW + 384 + EVIDENCE\_DELAY + REORG\_MARGIN)/12 \rceil + 2 < 8,191$; greater than every normal-context evidence and schedule path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

13 Production gates and verdict

The state-machine architecture is fixed enough for a prototype, but the repository is *not* an implementation-ready consensus specification. Its initial executable profile release bundle is absent. Before that status can change, one reviewed commit must contain all of the following normative artifacts and two independent implementations must reproduce every hash and execution result:

- canonical CBOR schema, exact profile bytes and `executionProfileHash`, plus a rejecting decoder corpus;
- an immutable manifest binding protocol version, chain IDs, activation, profile hash, verifier address/runtime hash and verifier-key hash;
- exact predeploy/proxy/router topology, terminal runtime code hashes, constructor immutables and relevant storage selectors for AnchorV4, InboxV2ActivationGate, InboxApply, SignalService and Bridge;
- exact nonzero `bytes32` Ethereum genesis/registry namespace, source-domain vector, EIP-2935 history lookup, canonical RLP and MPT decoders, append-only SourceVerifierRegistry domains/support/manifest authorization, historical verifier routing, BridgeEpochRegistry/CreditRecord ABI, runtime and storage layout, proof limits and valid/invalid source-witness corpus;
- exact unsigned system transaction type and reserved sender addresses; activation-latch and cutover proof; system nonces; expanded BridgeV2 and bounded SignalService batch ABI, selectors and storage; fee handling, gas ceilings and all fork and header recurrence rules; and
- complete positive and negative vectors for parent, prestate, input, transactions, receipts, headers and poststate, as listed in §11, including source-domain changes, Bridge epoch and implementation rotation, unauthorized clones, pending-capacity races, PREACTIVE rejection, legacy preactivation attempts, first-block latch activation, orphan/replayed pending with a refreshed witness, durable-record absence, old-domain routing, topology support authorization and lifecycle, multi-credit batches and every V1/V2 replay path.

Those values are outputs of real compiled contracts, fork code and a real circuit/verifier key. Inventing placeholders in LaTeX or Python would make the profile hash look precise while leaving clients unable to construct the same escape block. This is missing normative consensus, not an empirical launch gate. Consequently a sound implementation-ready artifact cannot be completed by document-only revision; implementation and circuit work must now produce the bundle. Once it exists and is re-audited, production deployment remains blocked until every additional gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review.
2. **Contract gas.** `sealWindow`, `postData`, normal submit, `sync`, source-credit preparation/topology proof, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. Fuzzing includes empty registry, zero seat, full data session, full queue prefix and maximum history age.

3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, EIP-2935/RLP source-header and MPT proofs, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix D remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks and three independently administered canonical archives, plus public data-session mirrors, schedule sealers and recovery bots, complete a shadow-fork outage exercise. One archive serves a clean node, after deleting its state, body, receipt *and code* databases, from only the latest finalized package while the other two are removed. The exercise also removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality. Loss and restore drills verify that unavailable prestate is never misreported as bounded recovery. For every supported source domain, an analogous drill starts from a clean source node, reconstructs a fresh witness to an old immutable credit record, prepares a kind-1 credit, orphans its unfinalized destination transaction, then rebuilds the proof against a later depth-final header and replays it. It also tests both witness-age bounds and proves that a superseding destination profile cannot remove the old topology entry.
7. **External review.** Separate proof-system, Solidity, bridge and economic audits close all critical/high findings; the migration rehearsal and emergency governance path are in scope.

Verdict. *The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies, fixes a single consensus path, binds bridge credits to their source application, and defines bytecode-complete recovery packages. No additional critical/high architecture defect is known under the stated L1/source-chain, proof, funding, canonical-prestate and source-state-witness availability assumptions. Nevertheless the design is* not implementation-ready*: the initial executable profile bundle above does not exist. This audit therefore stops at the document-only impossibility boundary, instead of falsely promoting test fixture hashes into production consensus.*

A Normative commitment encodings

All integers in packed commitments are unsigned big-endian at the named width; addresses are 20 bytes, hashes are 32 bytes, and ASCII domains have no length prefix or trailing NUL. `H` is Ethereum legacy Keccak-256. No `abi.encodePacked` call may contain a dynamic field. Raw transaction bytes are represented by `H(rawTx)` plus an explicit `u32(byteLength)`.

The canonical core/base, candidate, schedule list, sealed-session list and execution outputs use these exact encodings:

```
coreHash = H("slot-chain-core-v2" || u64(l2BlockNumber) || tipHash ||
  u64(tipSlot) || stateRoot || u64(messageCursor) || winningDataCommitment ||
  u256(nextBaseFee) || u64(nextExcessBlobGas))
baseCanonicalHash = H("slot-chain-canonical-v2" || coreHash ||
  u64(canonicalizedAtBlock))
H("slot-chain-candidate-v2" || baseCanonicalHash || u16(n) ||
  (u64(slot) || blockStructHash || blockHash || bodyRoot ||
   dataManifestRoot || u64(messageEnd))*)
H("slot-chain-schedule-list-v1" || u8(count) ||
  (u64(window) || entryRoot || seed)*)
H("slot-chain-session-list-v1" || u8(count) ||
  (sessionId || u16(recordCount) || root)*)
winningDataCommitment = H("slot-chain-winning-data-v1" ||
  candidateCommitment || sessionRefsCommitment)
H("slot-chain-outputs-v1" || stateRoot || transactionsRoot || receiptsRoot ||
  logsBloomHash || withdrawalsRoot)
```

Here $n \leq 4096$; schedule windows are ascending and sessions are ascending by ID. `blockStructHash` is the EIP-712 struct hash of the exact signed `SlotChainBlock` fields in §5.1; tier-3 recovery uses the same struct fields without a signature. These hashes are respectively the statement's `baseCanonicalHash`, `candidateCommitment`, `scheduleRootsCommitment`, `sessionRefsCommitment` and `executionOutputsCommitment`. Settlement must recompute the displayed `winningDataCommitment` before verifier dispatch and again before the canonical write; unsorted or duplicate lists reject.

The 64-cell registry leaf at index i is

```
H("slot-chain-registry-leaf-v1" || u8(i) || u8(present) ||
  address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || bytes32(trancheRoot) || u64(tombstonedAtL2Slot))
```

An empty cell has `present=0` and every following byte zero. A present cell has `present=1`; `UINT64_MAX` means not tombstoned. At tree height $h = 0, \ldots, 5$, the parent is:

```
H("slot-chain-registry-node-v1" || u8(h) || left || right)
```

There is no padding because there are exactly 64 leaves.

Admission position $i < 1,136$ is:

```
H("slot-chain-admission-leaf-v1" || u16(i) || u8(present) ||
  u8(location) || address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || u64(tombstonedAtL2Slot))
```

Location is 1 for active and 2 for liability; an empty leaf has present and all following fields zero. Positions $1,136 \ldots 2,047$ and unused positions are canonical empty leaves. Its eleven levels use `H("slot-chain-admission-node-v1"||u8(height)||left||right)`. `admissionVersion` increments before publishing the corresponding root; the pair is stored together and never reconstructed from a newer root. Tranche roots occur only in the active registry and liability storage records, not in this stable signer-identity commitment.

The omission-free ranked-entry commitment for window W has exactly 64 leaves:

```
H("slot-chain-entry-leaf-v1" || u8(rank) || u8(present) || address20 ||
  u192(bond) || u64(registrationIndex) || u64(effectiveL2Slot) ||
  u64(tombstonedAtL2Slot) || bytes32(trancheLeafHash))
```

followed by six levels of `H("slot-chain-entry-node-v1"||u8(height)||left||right)`. Empty ranks have all fields zero. The schedule circuit derives all 384 tickets from this root and seed; a schedule list cannot substitute a different set.

Each registry cell's tranche ring is a 512-leaf tree. Leaf index j is:

```
H("slot-chain-tranche-leaf-v1" || u16(j) || u64(window) || u8(state) ||
  u192(amount) || u64(liableUntil))
```

An empty leaf has `window=UINT64_MAX`, state `EMPTY=0` and remaining fields zero. Other states are `FREE=1`, `RESERVED=2`, `LIABLE=3`, `RELEASED=4` and `SLASHED=5`. A ring-index collision may be overwritten only from RELEASED or SLASHED after its absolute release predicate; the write installs the new exact window and FREE/RESERVED state atomically. `liableUntil=0` for EMPTY/FREE; reservation sets it exactly to `evidenceDeadline(window)+REORG_MARGIN_SECONDS`, and LIABILITY preserves that value. RELEASED/SLASHED preserve it until safe overwrite. Its nine node levels use `H("slot-chain-tranche-node-v1"||u8(height)||left||right)`. The snapshot supplies one inclusion proof for index `Wmod512` for every active cell, including canonical empty and `FREE` leaves.

Kind-0 forced leaf i is legacy Keccak over this exact packed sequence:

```
"slot-chain-force-user-v2" || u32(i) ||
address20(sender) || u64(nonce) || u256(l2ChainId) || bytes32(rawTxHash) ||
u32(byteLength) || u64(gasLimit) || u64(accountedGas) ||
u256(maxFee) || u64(validUntil) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

A kind-1 leaf is:

```
"slot-chain-force-bridge-v7" || u32(i) || msgHash ||
u256(srcChainId) || sourceDomainId || u64(srcEpoch) || address20(srcBridge) ||
u64(emittedAtBlock) || u64(srcBlockNumber) ||
srcBlockHash ||
u256(destChainId) ||
address20(srcOwner) ||
address20(destOwner) || u256(value) || calldataHash || escrowId ||
u32(byteLength) || u64(accountedGas) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

Its application has no expiry; no `validUntil` field exists.

For `forcedDescriptorCommitment`, `descriptorBytes` is exactly the corresponding leaf sequence beginning immediately after `u32(i)`; the list already supplies index and kind, so neither leaf ASCII domain nor index is duplicated inside it. Its fixed byte length is 220 for kind 0 and 420 for kind 1. The list encoding and optional first excluded boundary are defined in §8. The circuit prepends the correct leaf domain and index to reconstruct every Merkle leaf; any other length rejects.

`canonicalTopologyBytes` has this exact packed grammar:

```
u8(1) || bytes4(entrySelector) || u8(nodeCount) ||
(u8(kind) || address20(account) || runtimeCodeHash || u8(slotCount) ||
   (bytes32(slot) || bytes32(value))*)*
```

Here `1<=nodeCount<=8`, `slotCount<=8`, and the complete encoding is at most 4,096 bytes. Kinds are 0=terminal, 1=EIP-1967 proxy, 2=profile beacon and 3=profile fork router. Exactly the last node is terminal; accounts are nonzero and unique; code hashes are nonzero; and slots within a node are strictly increasing and unique. The first account is `srcBridge`. The immutable profile maps every admitted `(kind,runtimeCodeHash,entrySelector)` to its exact required slot set and a total next-account extractor. Verification proves every listed account, code hash and slot/value under the one source state root, applies each extractor to obtain exactly the following account, and ends at the terminal node. It rejects an unknown tuple, missing/extra slot, duplicate, cycle, unconsumed node, nonterminal final node or bytes absent from the profile's finite exact allowlist. Thus "complete topology" is executable matching against reviewed descriptors, not bytecode introspection or a caller-chosen list.

The depth-32 vector empty leaf is `H("slot-chain-force-empty-v2")`. A parent at height $h = 0,\ldots,31$ is `H("slot-chain-force-node-v2"||u8(h)||left||right)`. The queue commitment is `H("slot-chain-force-root-v2"||u64(count)||treeRoot)`; leaves at indices `count..2^32-1` are empty. The canonical range proof recursively visits the smallest tree covering `[start,end]` (including a boundary leaf when `end<count`); it emits in DFS order exactly one hash for each maximal outside subtree. The verifier rejects extra/missing nodes, a nonempty padding leaf, or a reconstructed root/count mismatch. Append at old index i starts with the new leaf at height zero, repeatedly hashes `frontier[h]` on the left while bit h of i is one, stores the carry in the first zero-bit frontier position, increments count, and reconstructs all higher right subtrees from `FORCE_EMPTY[h]`. This is the same root as the full vector and uses exactly 32 frontier words.

A data-chunk leaf is:

```
H("slot-chain-data-leaf-v1" || bytes32(sessionId) || u16(index) ||
  bytes32(versionedHash) || bytes32(fullBodyRoot) || u16(blockOrdinal) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  bytes32(chunkRoot) || address20(publisher) ||
  u64(validUntil) || u256(z) || u256(y))
```

MMR append merges equal-height peaks as `H("slot-chain-data-node-v1"||u8(height)||left||right)`. Peaks are stored left-to-right in strictly decreasing height. The root bags them rightmost-to-leftmost as `H("slot-chain-data-bag-v1"||u16(count)||u8(peakCount)||(u8(height)||peak)*)`. An inclusion proof provides the leaf index, merge siblings bottom-up with a left/right bit at each height, then every other peak in this bag order; duplicate

heights, a count inconsistent with peak heights, or unused proof elements reject. The empty MMR is the bag hash with zero count and zero peaks.

A manifest leaf is:

```
H("slot-chain-manifest-leaf-v1" || u16(position) ||
  u16(blockOrdinal) || sessionId || u16(recordIndex) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  fullBodyRoot || chunkRoot)
```

Its next-power-of-two tree uses

```
H("slot-chain-manifest-node-v1" || u8(height) || left || right)
H("slot-chain-manifest-root-v1" || u16(count) || treeRoot)
```

for the node and final root respectively; padding uses the domain-separated empty leaf. For zero entries, `treeRoot=H("slot-chain-manifest-empty-v1")`. The zero-discretionary- transaction body is the ordinary body encoding `u32(0)` and therefore `H("slot-chain-body-v1"||u32(4)||u32(0))`. Tier 3 requires exactly those empty body/manifest values and the zero-count session-list commitment. This tree is instantiated separately for every block. `position` starts at zero inside that block; `blockOrdinal` remains the candidate-wide block index. A two-block candidate therefore commits two signed manifest roots and never hashes their leaves into one shared tree. The InboxApply disposition commitment is:

```
H("slot-chain-dispositions-v1" || u64(start) || u64(end) ||
  u16(count) ||
  (u64(queueIndex)||u8(disposition)||u32(txIndex)||resultHash)*)
```

Here `UINT32_MAX` means no Ethereum transaction.

The recovery ID is the encoding in §7. Schedule leaf/root, seed and tie hashes are as in §4; body and blob encodings are as in §6. `commitment-model.py` is the normative executable fixture. Its initial vectors are:

```
TYPEHASH      ee6a8c8e31e8245cd527869508f6e464d6084893991203876f734d1855aed87c
registryRoot  0bf297d7b9b6a5529a319a06cb08484923a89bab15d51f8baeaf5c30bebdf3fd
admissionRoot 3bf2dcaf78292c832108e29205bf99cc2d22137a0545e4528d8da7309d4b482b
entryRoot     acee83a690b868a4a7960c55a9f7228f91cad26b704e24106d4db87e9c7a8f34
forcedRoot    ab03f105ee7d619fb2c81d31b38760720dff2cb35471bc28fdc063c31f71bd67
sourceDomain  0200a9c8b107399151be3a25b7ebfd4b500b616a3e9a7eab7698284ea0bfbb52
bridgeExec    902970f59bbc440b3d36429ad2add690959871287a52e65123001a635404c84b
bridgeLeaf    25166869aba48ca26b8ccb87e0172605eee3f191be9aaae8aca0b35f8b88f316
creditId      4e5ca7e70f7832fd08235821dd0e4b54c05635ab01eb2ce24562d77871307ad5
escrowId      9602fc9d79915d2e8fa052d9be72069ccbf13839b406584daf07088b34b648c5
inboxSlot     81cea6e71e51e82a170dbdcf00536cba7ecf493af91ac26b8bbf577fbf6f6d05
failedSignal  041527ac55aff43ae75e842b2c9ea7be33eab83fccb43c6813e1187af5ce5194
bridgeResult  c23272f6afd7c21ba7bc1ce211bd0b7caf5a498ffa782c63dbe3181768e82684
manifestRoot  417be737a57e38eb410f2d6e65c77ee19d5c314cdaf432067861c6a36c6a990f
normalContext 441dee77fdcd856bfb41504fdcbcbd97de481a881e53017a5983d1210f6d22a8
migrationData 916b1b24deee7fde4484c3cebe42937c32b41deb7b39529feb517986e7414399
winningData   d28b2d74341e9c5d90835e5051b2fc8189e1814f178d9580f4e9f66a88cfa631
```

```
statementHash 5bd1192b005ebce2f290d11924d3e9548a18e12e72aa4626d8397a85ad36f8f7
recoveryId    84ea35ee4c31975fc4499964ea6d1f72abc6291666c89dc2ef1a61447644103b
bodyRoot      0f4e161a46c8b18c2a86f23a0a4e7169a838a12af8b389f65e97b547a99707e9
```

Release CI must reproduce every vector in Solidity, Go, Rust and the circuit.

B Normative transition ordering

Every external candidate entry point implements the following structure:

```
function armNormalContext() returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;
    if (mode == NORMAL_ARMED && block.number > armBlockNumber + 255) {
        armBlockNumber = block.number;
        emit NormalContextRearmed(armBlockNumber);
        return REARMED;
    }
    if (mode != NORMAL_IDLE) return IGNORED;
    armBlockNumber = block.number; // no caller-selected hash or anchor
    mode = NORMAL_ARMED;
    return ARMED;
}

function activateNormalContext(armHeaderRlp) returns (Status) {
    if (sync()) return SYNCED;
    if (mode != NORMAL_ARMED) return IGNORED;
    age = block.number - armBlockNumber;
    require(age > 0 && age <= 255);
    armBlockHash = blockhash(armBlockNumber);
    require(keccak256(armHeaderRlp) == armBlockHash);
    freeze base, stable admission pair and parsed arm header;
    freeze minimum slot, deadline and landing horizon;
    mode = NORMAL_OPEN;
    return ACTIVATED;
}

function submit(input) returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;     // successful transaction, never revert
    if (mode == NORMAL_OPEN) return submitNormal(input);
    if (mode != RECOVERY) return REJECTED_NO_CONTEXT;
    return submitRecovery(input);
}

function sync() returns (bool changed) {
    if (mode == RECOVERY && now > round.expiresAt) {
        rollRound();               // same episode/base/burn; new target
```

```
        return true;
    }
    if (mode in {NORMAL_ARMED,NORMAL_OPEN} && forcedHeadDue &&
        (no deadline || normal window immature)) {
        cancel normal window;
    } else if (normal deadline matured) {
        if (best exists) {
            liveBoundaryDueAt = ForcedQueue.dueAt(best.endCursor);
            dataLive = now + REORG_MARGIN_SECONDS <= best.minDataExpiry;
        }
        if (best exists && liveBoundaryDueAt > now && dataLive)
            commit(normalBest);
        else cancel normal window;
    }
    if (isNormalMode(mode) &&
        (finalLagBreach || forcedHeadDue)) {
        openRecoveryEpisodeAndRound();
        if (finalLagBreach) terminate incumbent once;
        return true;
    }
    return changed;
}
```

No transfer, reward calculation, unbounded loop or caller-controlled external call occurs in canonical settlement. Optional reward materialization is a later transaction in a separate module.

C Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Only a mature best that covers the required forced prefix closes; immature state is canceled.
4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Reward materialization is a separate non-authoritative transaction.
5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.
7. **Arithmetic snapshot slot without EIP-4788 parent semantics.** Rejected. A bounded for-

ward carrier search authenticates the parent source or fails closed.

8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.

D Executable consensus models

`lookahead-model.py` reports 36 assertions over legacy Keccak, EIP-4788 carrier/parent selection, immutable seal deadlines, post-seal tombstones, partial/empty registry sealing, eligibility-before-ranking, capped D'Hondt and feasible placement, including absolute clock conversion, execution-block finality depth, fixed seed and complete-schedule vectors.

`settlement-window-model.py` reports 123 assertions over PREACTIVE/migration, early-return semantics, force-due precedence, renewable recovery, no-seat escape, exact slot/header time, durable-descriptor prefix predicates, immutable anchor chronology, O(1) due boundaries, late close, sealed data sessions, frozen admission pairs, bounded replacement/liability generations, tranche release units, bridge reservations, L2 activation gating, refreshable source witnesses, manifest-authorized historical source support, replay and timing geometry. Cryptographic booleans in that model are not cryptographic evidence.

`commitment-model.py` reports 84 vectors/properties for exact EIP-712 domain/struct/digest, canonical and statement hashes, registry/admission/entry/ tranche commitments, forced descriptors/vector/range proofs, session MMR, per-block manifest, source domains/topology, emission/witness separation, credit/escrow/inbox/failure slots and atomic ordered batches, dispositions, recovery ID, Fiat–Shamir input, body chunks and blob codec. The valid KZG-opening, signature-recovery, independent-implementation and proof gates remain mandatory.

E Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode/revision, the current base tuple, one stored prior-block anchor at required depth and one frozen queue target. Expired rounds are permissionlessly renewable without another burn.
5. A due forced head advances in the next canonical block; appends omitted by a frozen recovery round cannot become due until that round expires. Prefix count, bytes and accounted gas are independently bounded.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every block header timestamp equals genesis plus its signed slot. Every tier-1/2 block has a valid scheduled signature under the frozen admission version/root pair, strict slot progress and the candidate's one authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. A tombstoned generation cannot re-enter while retained; safe later address reuse receives a fresh registration index only after every evidence deadline. Each window has an independent slashable tranche and finite release state.
11. Candidate, window, data-session, Merkle range proof, queue count/bytes/gas, history and storage-retention work is bounded by consensus constants and safe eviction.

12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.
13. After ordinary quiescence, migration freezes old ingress, imports canonical/bridge cursors into PRE-ACTIVE settlement and switches the one active queue router atomically. V2 L2 entry points remain disabled under legacy rules; the first custom post-cutover Anchor transaction activates their shared gate.
14. A kind-1 credit is prepared only from a bounded, depth-final, EIP-2935-authenticated source header proving an append-only registry credit, its immutable epoch and manifest-authorized historical domain/topology support. The credit record is the source signal; no mutable external slot is required. The witness header is refreshable and is distinct from `emittedAtBlock`. `PENDING` reserves capacity and validates every static append predicate; a reorged preparation can be re-proved under a later header, and source topology support cannot be removed.